



Bootcamp: Engenheiro(a) de Dados

Atividade modular - Enunciado

Módulo 2: Pipeline de Dados

Objetivos de Ensino

Exercitar os seguintes conceitos vistos em aulas gravadas:

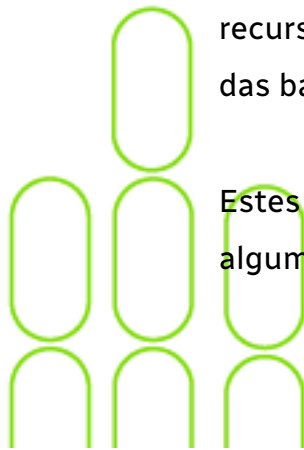
1. ETL.
2. Ferramentas ETL – Pentaho PDI e Apache Hop.
3. Processos de carga.
4. Orquestração.

Enunciado

1º Parte

Vamos utilizar dois arquivos de entrada com informações da Capes onde é feito uma avaliação da Pós-Graduação Stricto Sensu. A avaliação da pós-graduação stricto sensu tem como objetivo a certificação da qualidade da pós-graduação brasileira bem como a identificação de assimetrias regionais e de áreas estratégicas do conhecimento. Ela é orientada pela Diretoria de Avaliação/CAPES e realizada com a participação da comunidade acadêmico-científica por meio de consultores ad hoc. Tem como pilares a formação pós-graduada de docentes para todos os níveis de ensino e de recursos humanos qualificados para o mercado, bem como o fortalecimento das bases científica, tecnológica e de inovação.

Estes dois arquivos servirão como entradas de dados para executarmos algumas atividades de transformação no Pentaho PDI.





Posteriormente, migraremos a transformação feita no Pentaho PDI para o Apache Hop.

2º Parte - opcional, mas muito interessante para aprendizado

Ter um primeiro contato com atividades de orquestração de dados em nuvem pelo uso do Apache Airflow.

O Apache Airflow é uma plataforma de fluxo de trabalho e orquestração de código aberto que permite agendar, agilizar e monitorar atividades complexas de processamento de dados. Ele é frequentemente usado para criar, agendar e monitorar fluxos de trabalho de processamento de dados em ambientes de nuvem.

Objetivos

Executar as práticas no Pentaho PDI e, posteriormente, transportar para o Apache Hop.

Não é preciso entregar nada. As respostas às práticas irão suportar vocês nas respostas do questionário do trabalho prático.

Opcionalmente, subir uma instalação do Apache Airflow e orquestrar a prática da transformação por ele.

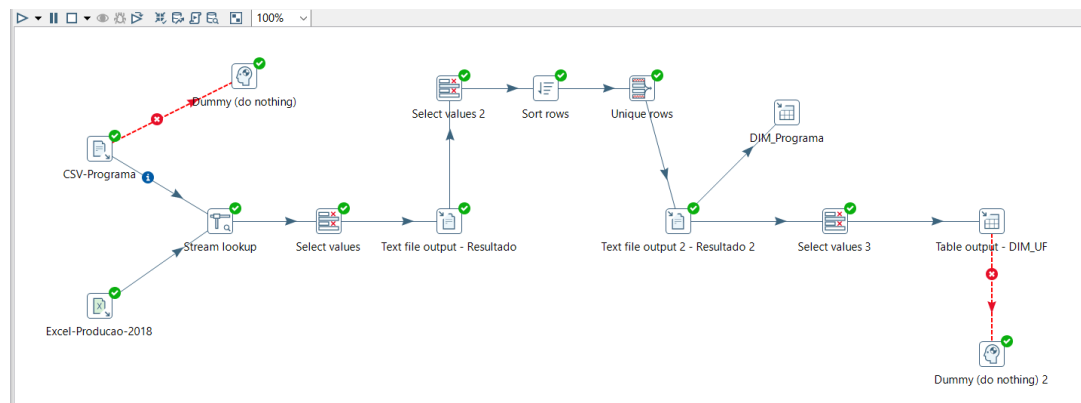


1º Parte

Atividades

Basicamente iremos construir uma transformação que ao final irá alimentar uma tabela Dimensão chamada Dim-Produção.

1) Prático Pentaho PDI



Atividades:

Processo de carga;

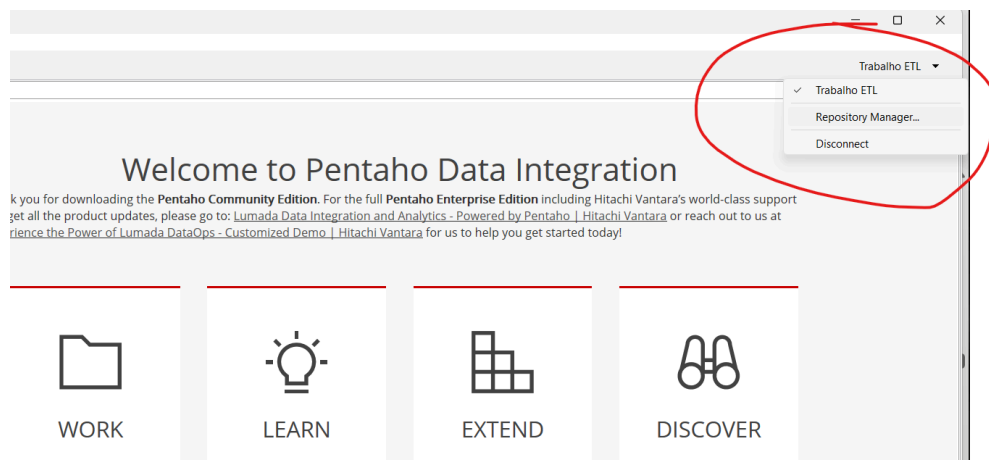
O primeiro passo é baixar os arquivos do link abaixo ou pegá-los na plataforma do IGTI:

https://drive.google.com/drive/folders/12x_QtfqdyL6b3vvUj9XMu4M2DsNCZdVF

- Producao-2018-Bibliografica-Artigo.xls.
- Programa-2018-Capes.csv.

Para iniciar, abra o Pentaho PDI (Spoon.bat).

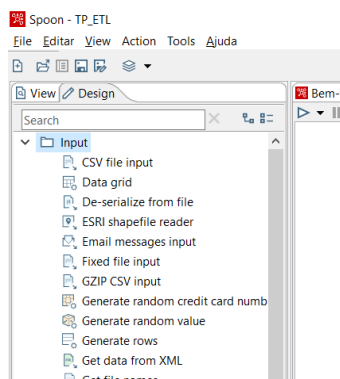
Vamos criar o repositório apontando para o diretório do nosso projeto:



Depois vamos criar uma transformação, um arquivo .ktr:

(menu *File | Novo | Transformação*)

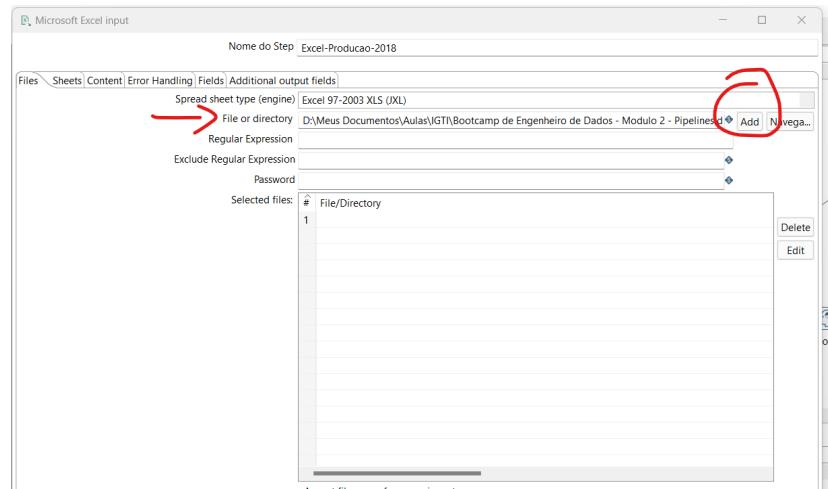
Abra a categoria input e selecione os Steps *Excel Input* e *CSV file Input*.



Edite o Step Microsoft Excel input (grupo input no Design) com os seguintes parâmetros:

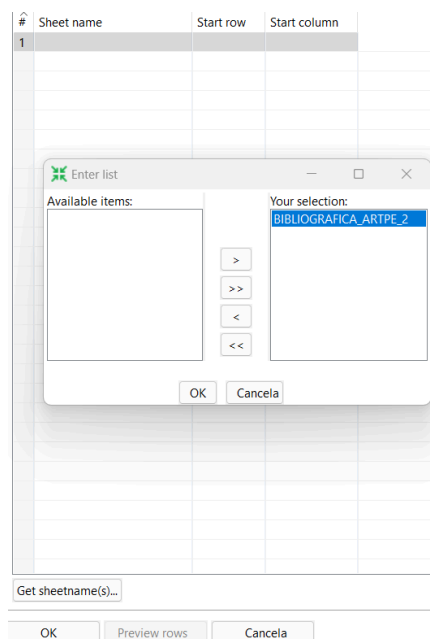
Aba Files

- **File or directory:** informe o arquivo Producao-2018-Bibliografica-Artigo.xls. Para ter certeza de que o arquivo foi localizado clique no botão show filename para certificar que ele está sendo exibido.



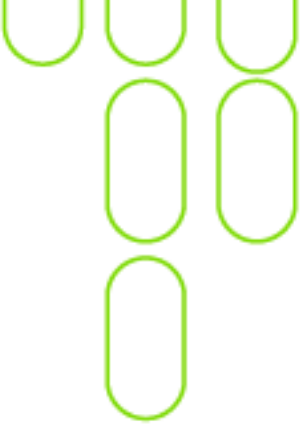
Aba Sheets

- Clique no botão Get sheetname e escolha a planilha desejada.



Aba Content

- Certifique-se que o campo Header esteja marcado.
- Encoding: para aceitar a acentuação sem mostrar como caracteres especiais, em geral, o mais utilizado seria o UTF-8,



visto que suporta todo o alfabeto e acentos além de uma gama gigantesca de caracteres especiais, mas isso pode variar e vamos usar o ISO-8859-1.

- o Mude o campo **Encoding** para **ISO-8859-1**.

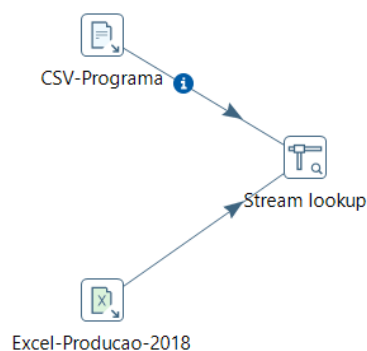
Aba Fields

- Clique no botão Get fields from header now e veja todos os campos disponíveis.
- Dê uma olhada nos dados que serão extraídos do arquivo clicando no botão Preview rows. Clique Ok e salve a transformação.

Edite o Step CSV file input com os seguintes parâmetros:

- Filename: localize o arquivo Programa-2018-Capes.csv com o botão Navegar.
- Delimiter: informe: “;” (ponto e vírgula).
- Mude o campo Encoding para ISO-8859-1.
- Desmarque a opção Lazy conversion.
- Clique no botão Obtem campos e veja os campos que serão lidos.
- Clique no botão Preview para visualizar uma amostra dos dados.
- Retire o símbolo da moeda (R\$) da propriedade Currency. Basta apagar essa informação em todos os atributos.
- Confirme se o Header row present está marcado. Isso indica que o arquivo tem um header com o nome dos campos.

Adicione o Step Stream lookup e faça as ligações conforme figura abaixo.



Selecione o Step Stream lookup com os seguintes parâmetros:

- Lookup Step: escolha o Step do CSV.
- Clique nos botões Get fields e Get lookup fields. O lookup fará a ligação dos arquivos pela chave comum. É uma ligação de 1 para N, ou seja, um Programa tem N Publicações e a ligação entre os arquivos se dá pelo campo CD_PROGRAMA_IES. Temos esse campo nos dois arquivos e é o identificador do Programa.

Stream lookup

Step name: Stream lookup

Lookup step: CSV-Programa

The key(s) to look up the value(s):

#	Field	LookupField
1	CD_PROGRAMA_IES	CD_PROGRAMA_IES

Specify the fields to retrieve :

#	Field	New name	Default	Type
1	AN_BASE			Integer
2	NM_Grande_Area_Conhecimento			String
3	NM_Area_Conhecimento			String
4	NM_Area_Basica			String
5	NM_Subarea_Conhecimento			String
6	NM_Especialidade			String
7	CD_Area_Avaliacao			Integer
8	NM_Area_Avaliacao			String
9	CD_Entidade_Capes			Integer
1..	CD_Entidade_Emec			String
1..	SG_Entidade_Ensino			String
1..	NM_Entidade_Ensino			String
1..	NM_Regiao			String
1..	SG_UF_Programa			String
1..	NM_Municipio_Programa_Ies			String
1..	NM Modalidade_Programa			String

Preserve memory (costs CPU) ☒

Key and value are exactly one integer field ☐

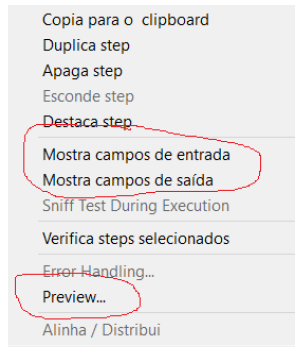
Use sorted list (i.s.o. hashtable) ☐

Help OK Cancela Get Fields Get lookup fields

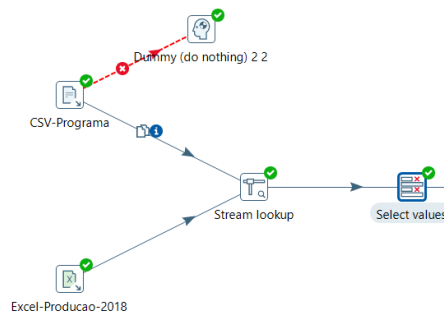
Na grade de cima, remova todos os campos deixando apenas o campo CD_PROGRAMA_IES.

Na grade de baixo deixe os campos que deseja que sejam mostrados a partir da junção.

Clique com o botão direito em cima do Step Stream lookup e escolha as opções abaixo de visualização.



Suponha que não precisamos de todos os campos vindos da junção e para isso é necessário inserir o Step Select values fazendo a ligação necessária.



Aba Select & After

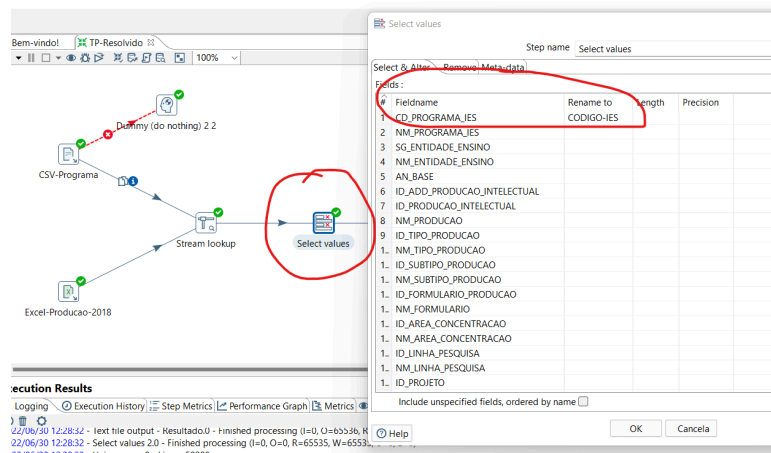
- mostra os campos após você clicar no botão Get fields to select.

Aba Remove

- mostra os campos após você clicar no botão Get fields to remove. Os campos que permanecerem nessa lista serão removidos.

Ainda no Step Select values

- você vai renomear o campo CD_PROGRAMA_IES para CODIGO-IES.



Insira o Step Text file output com os seguintes parâmetros:

Aba File

- **Filename:** caminho onde irá salvar o arquivo + nome do arquivo txt.

Aba Fields

- Clique no botão **Obtém campos** e veja os campos que serão gravados.
- Você pode fazer alterações diretamente na grade. Clique no botão **Minimal width** e veja que o Step fornece um formato padrão para os campos.

Rode o arquivo de transformação e veja se gravou o arquivo texto no diretório indicado.

Observe que inicialmente dará um erro referente a linha 878. Na realidade teremos erro em duas linhas. Isso porque há uma tentativa de converter valor no campo **CD_CONCEITO_PROGRAMA** de String para Integer.

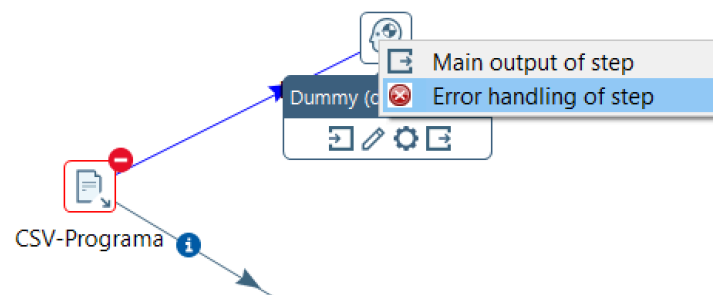
```

2020/07/26 15:32:25 - CSV-Programa.0 - ERROR (version 9.0.0.0-423, build 9.0.0.0-423 from 2020-01-31 04:53.04 by buildguy) : Erro inesperado
2020/07/26 15:32:25 - CSV-Programa.0 - ERROR (version 9.0.0.0-423, build 9.0.0.0-423 from 2020-01-31 04:53.04 by buildguy) : org.pentaho.di.core.excep
2020/07/26 15:32:25 - CSV-Programa.0 - There were 1 conversion errors on line 878
2020/07/26 15:32:25 - CSV-Programa.0 - Unexpected conversion error while converting value [CD_CONCEITO_PROGRAMA String] to an Integer
2020/07/26 15:32:25 - CSV-Programa.0 - CD_CONCEITO_PROGRAMA String : couldn't convert String to Integer
2020/07/26 15:32:25 - CSV-Programa.0 -

```

Você deve tratar o erro na transformação para que o processo não seja interrompido.

Insira o **Step Dummy (do nothing)** e ligue o **Step CSV** ao **Step Dummy**. Na ligação selecione **Error handling for Step**. Aparecerá uma caixa onde você deve aceitar a distribuição pelo botão Distribuir.



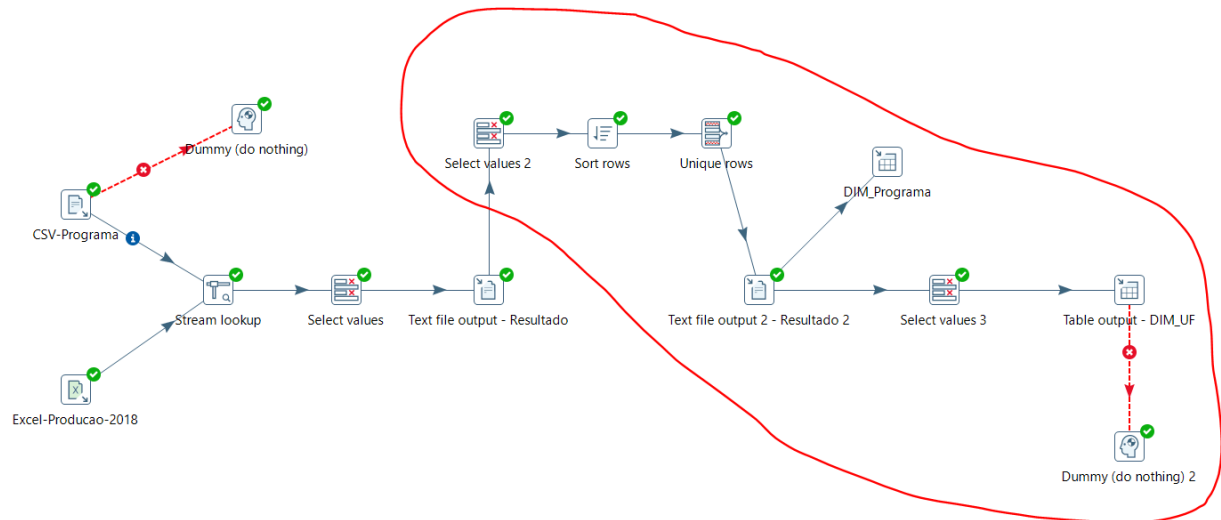
Rode novamente e veja que será concluído com sucesso.

Clique com o botão direito no **Step Dummy** e selecione **Preview**. Veja as linhas que deram erro.

Rows of step: Dummy (do nothing) (2 rows)

#	AN_BASE	NM_GRADE_AREA_CONHECIMENTO	NM_AREA_CONHECIMENTO	NM_AREA_BASICA	NM_SUBAREA_CC
1	2018	MULTIDISCIPLINAR	CIÊNCIAS AMBIENTAIS	CIÊNCIAS AMBIENTAIS	NÃO SE APLICA
2	2018	CIÊNCIAS HUMANAS	CIÊNCIA POLÍTICA	CIÊNCIA POLÍTICA	NÃO SE APLICA

Agora é com você. Veja a figura abaixo e termine o trabalho prático.



Observe o conteúdo de cada Step.

Step “Select values 2”

A saída desse Step é:

Step name:

Fields:

#	Fieldname	Type	Length	Precision	Step origin	Storage	Mask	Currency	Decimal	Group	Trim	Comments
1	CODIGO-IES	String	-	-	Select values	normal					nenhum	
2	NM_PROGRAMA_IES	String	-	-	Excel-Producao-2018	normal					nenhum	
3	SG_ENTIDADE_ENSINO	String	-	-	Excel-Producao-2018	normal					nenhum	
4	NM_ENTIDADE_ENSINO	String	-	-	Excel-Producao-2018	normal					nenhum	
5	NM_REGIAO	String	12	-	Stream lookup	normal		R\$	,	.	nenhum	
6	SG_UF_PROGRAMA	String	2	-	Stream lookup	normal		R\$	,	.	nenhum	
7	NM_MUNICIPIO_PROGRAMA_IES	String	21	-	Stream lookup	normal		R\$	,	.	nenhum	
8	NM_MODALIDADE_PROGRAMA	String	12	-	Stream lookup	normal		R\$	,	.	nenhum	

Step Sort rows

Sort rows

Nome do Step:

Sort directory:

TMP-file prefix:

Sort size (rows in memory):

Free memory threshold (in %):

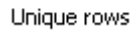
Compress TMP Files? ☐

Only pass unique rows? (verifies keys only) ☐

Fields:

#	Fieldname	Ascending	Case sensitive compare?	Sort based on current locale?	Collator Strength
1	CODIGO-IES	S	N	N	0

Step 1



Pré-Requisito: para entregar um resultado correto, o fluxo de entrada **deve ser classificado em uma etapa anterior**, caso contrário, apenas as linhas duplas consecutivas serão analisadas e filtradas. Para isso, vamos utilizar o **Step Sort rows**.

Step Text file output 2 – Resultado 2

Text file output

Nome do Step: Text file output 2 - Resultado 2

File Content Fields

Filename: D:\OneDrive\Aula\IGTI\ETL Bootcamp\TP e Desafio\TP\Resultado2

Pass output to servlet: ☐

Create Parent folder: ☒

Do not create file at start: ☐

Accept file name from field: ☐

File name field: [Empty]

Extensão: txt

Include stepnr in filename?: ☐

Include partition nr in filename?: ☐

Include date in filename?: ☐

Include time in filename?: ☐

Specify Date time format: ☐

Date time format: [Empty]

Show filename(s)...

Add filenames to result: ☒

Help OK Cancela

DIM_Producao (Step Table output)

Saída a Tabela

Nome do Step: DIM_Producao

Connection: Igti_MySql_3307

Target schema: tp

Target table: dim-programa

Commit size: 1000

Truncate table: ☐

Ignore insert errors: ☐

Specify database fields: ☒

Main options Database fields

Partition data over tables: ☐

Partitioning field: [Empty]

Partition data per month: ☒

Partition data per day: ☐

Use batch update for inserts: ☒

O nome da tabela está definido em uma: ☐

Coluna que tem o nome da tabela: [Empty]

Store the tablename field: ☒

Return auto-generated key: ☐

Name of auto-generated key field: [Empty]

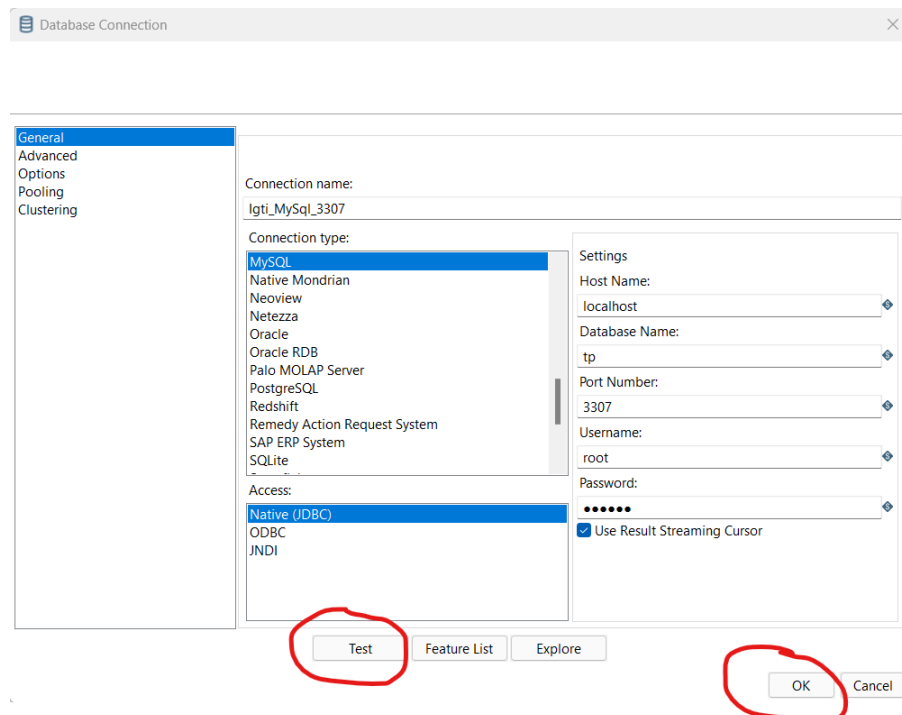
Help OK Cancela SQL

O **Step Table output** carrega dados em uma tabela do banco de dados. Esse Step é equivalente ao operador SQL INSERT e é uma solução quando você só precisa inserir registros. Se você quiser apenas atualizar linhas, use Update. Para executar os comandos INSERT e UPDATE, consulte o **Step Insert / Update**.

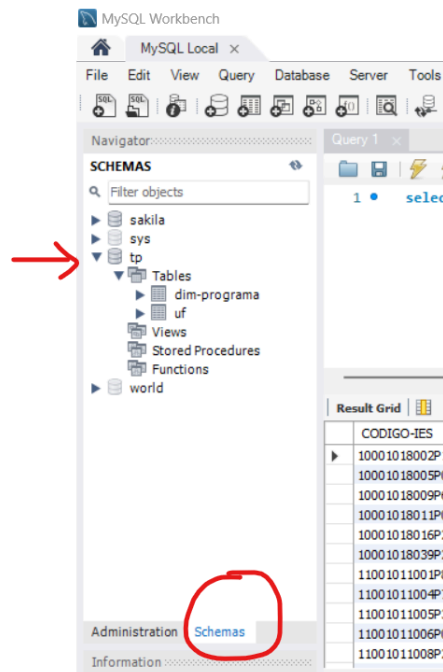
Esta etapa fornece opções de configuração para uma tabela de destino e opções relacionadas ao desempenho, como Commit Size e Use **batch update** para inserções. Existem configurações de desempenho específicas para um tipo de banco de dados que podem ser definidas nas propriedades JDBC da conexão com o banco de dados.

Use um gerenciador de Banco de Dados. No nosso caso poderia ser o MySQL Workbench ou MySQL Xampp, porém você pode usar outro banco de dados se conseguir estabelecer a conexão.

Connection: conexão com o banco de dados.

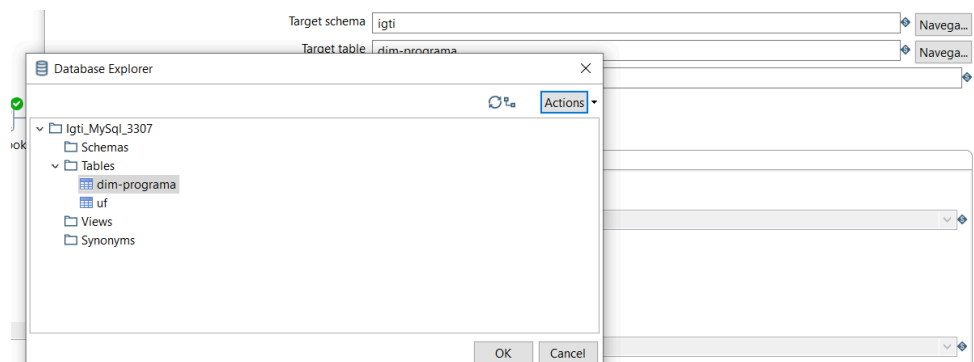


Target schema: nome do schema.

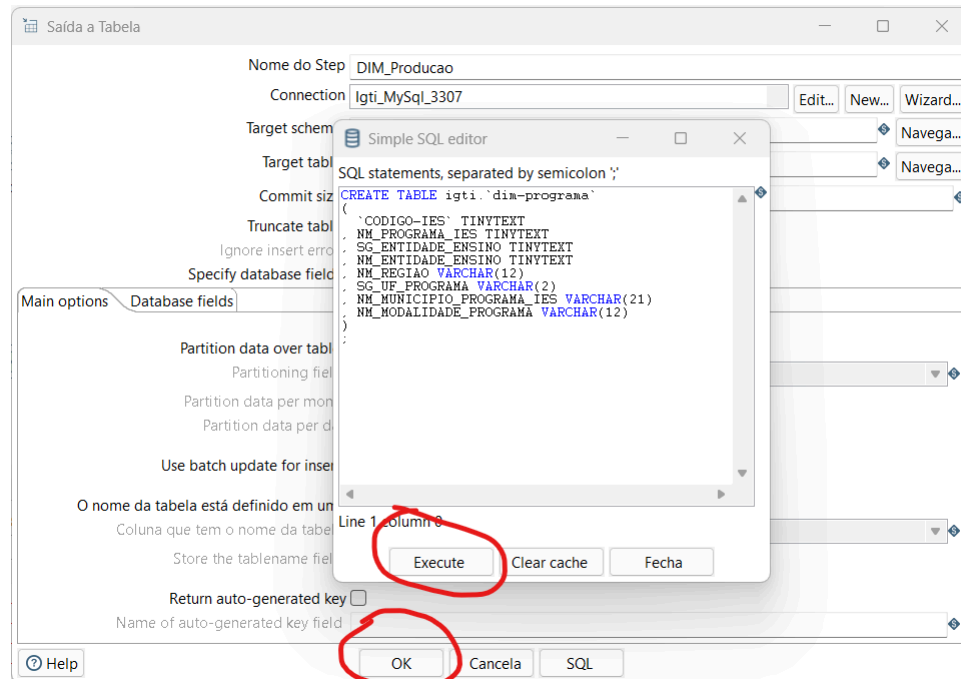


Target table: nome da tabela.

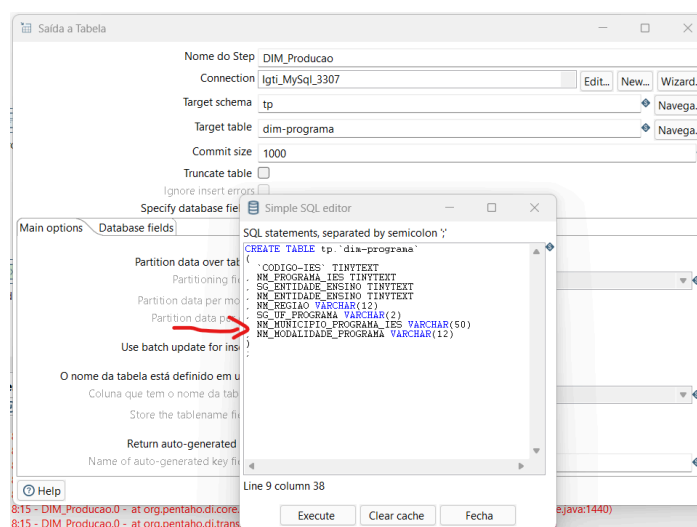
Para esse caso informe o nome da tabela e clique no botão Navegar. Se a conexão estiver ok, ele vai mostrar o caminho no banco de dados.



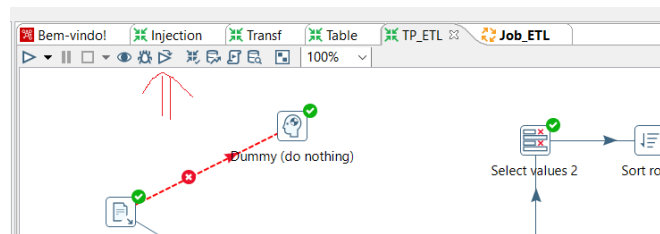
Se a tabela não existir ainda, clique no **botão SQL**. Irá abrir uma nova janela com o script de criação da tabela. Clicar no **botão Execute** e a tabela será criada, mas não o faça ainda. Cabe uma observação aqui.



O PDI se baseia nos primeiros registros do fluxo de dados para sugerir o script de criação da tabela no banco de dados. Portanto ele vai sugerir que o campo **NM_MUNICIPIO_PROGRAMA_IES** seja criado com **tamanho 21**. O problema é que mais ao final dos dados, nas linhas finais, tem registros onde esse campo **NM_MUNICIPIO_PROGRAMA_IES** tem um valor bem maior, **próximo de 50 caracteres**. Assim você precisa mudar esse tamanho para não receber erros durante a transformação.



Rode a transformação.



Clique no **Step DIM_Producao** com o botão direito e selecione **Preview** para ver os dados.

Examine preview data

Rows of step: DIM_Programa (1000 rows)

#	CODIGO-IES	NM_PROGRAMA_IES	SG_ENTIDADE_ENSINO	NM_ENTIDADE_ENSINO	NM_REGIAO	SG_UF_PROGRAMA
1	10001018002P1	BIOLOGIA EXPERIMENTAL	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO
2	10001018005P0	GEOGRAFIA	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO
3	10001018009P6	PSICOLOGIA	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO
4	10001018011P0	EDUCAÇÃO	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO
5	10001018016P2	EDUCAÇÃO ESCOLAR	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO
6	10001018039P2	DIREITOS HUMANOS E DESENVOLVIMENTO DA JUSTIÇA	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO
7	11001011001P8	ECOLOGIA E MANEJO DE RECURSOS NATURAIS	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC
8	11001011004P7	PRODUÇÃO VEGETAL	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC
9	11001011005P3	SAÚDE COLETIVA	FIOCRUZ	FUNDACAO OSWALDO CRUZ (FIOCRUZ)	NORTE	AC
1.	11001011006P0	CIÊNCIA, INOVAÇÃO E TECNOLOGIA PARA A AMAZÔNIA	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC
1.	11001011008P2	SANIDADE E PRODUÇÃO ANIMAL SUSTENTÁVEL NA AMAZÔNIA OCIDENTAL	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC
1.	11001011071P6	CIÊNCIA FLORESTAL	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC
1.	12001015002P7	QUÍMICA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
1.	12001015006P2	FÍSICA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
1.	12001015008P5	GEOCIÊNCIAS	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
1.	12001015012P2	INFORMÁTICA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
1.	12001015014P5	SOCIEDADE E CULTURA NA AMAZÔNIA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
1.	12001015016P8	CIÊNCIAS FLORESTAIS E AMBIENTAIS	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
1.	12001015021P1	ENGENHARIA ELÉTRICA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
2.	12001015023P4	HISTÓRIA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
2.	12001015025P7	CIÊNCIAS PESQUEIRAS NOS TRÓPICOS	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
2.	12001015027P0	SERVIÇO SOCIAL	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
2.	12001015032P3	CIÊNCIAS DA COMUNICAÇÃO	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
2.	12001015034P6	IMUNOLOGIA BÁSICA E APLICADA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM
2.	12001015037P9	PSICOLOGIA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM

Close Stop Get more rows

Rode o comando SQL abaixo no Banco de Dados para verificar os dados que foram inseridos na tabela.

```
select * from tp.`dim-programa`;
```

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

igti

Tables

dim-programa

uf

Views

Stored Procedures

Functions

mydb

sys

Administration Schemas

Information

Schema: igti

Query 1 SQL File 5*

Limit to 1000 rows

1 select * from igti.'dim-programa';

2

Result Grid

Filter Rows

Exports

Wrap Cell Contents

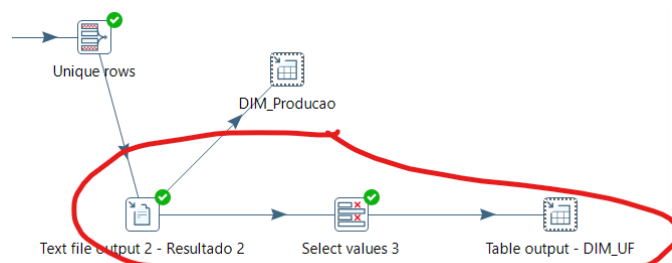
Fetch rows

CODIGO-JES	NM_PROGRAMA_JES	SG_ENTIDADE_ENSINO	NM_ENTIDADE_ENSINO	NM_REGIAO	SG_UF_PROGRAMA	NM_MUNICIPIO_PRC
10001018002P1	BIOLOGIA EXPERIMENTAL	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO	PORTO VELHO
10001018005P0	GEOGRAFIA	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO	PORTO VELHO
10001018009P6	PSICOLOGIA	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO	PORTO VELHO
10001018011P0	EDUCAÇÃO	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO	PORTO VELHO
10001018015P2	EDUCAÇÃO ESCOLAR	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO	PORTO VELHO
10001018039P2	DIREITOS HUMANOS E DESENVOLVIMENTO DA ...	UNIR	UNIVERSIDADE FEDERAL DE RONDÔNIA	NORTE	RO	PORTO VELHO
11001011001P8	ECOLOGIA E MANEJO DE RECURSOS NATURAIS	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC	RIO BRANCO
11001011004P7	PRODUÇÃO VEGETAL	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC	RIO BRANCO
11001011005P3	SAÚDE COLETIVA	FIOCRUZ	FUNDACAO OSWALDO CRUZ (FIOCRUZ)	NORTE	AC	RIO BRANCO
11001011006P0	CIÊNCIA, INOVAÇÃO E TECNOLOGIA PARA A A...	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC	RIO BRANCO
11001011008P2	SANIDADE E PRODUÇÃO ANIMAL SUSTENTÁVE...	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC	RIO BRANCO
11001011071P6	CIÊNCIA FLORESTAL	UFAC	UNIVERSIDADE FEDERAL DO ACRE	NORTE	AC	RIO BRANCO
12001015002P7	QUÍMICA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM	MANAUS
12001015006P2	FÍSICA	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM	MANAUS
12001015008P5	GEOCIÊNCIAS	UFAM	UNIVERSIDADE FEDERAL DO AMAZONAS	NORTE	AM	MANAUS

dim-programa 21 x

Agora vamos carregar uma dimensão UF a partir dos dados do arquivo de Resultado 2.

- Select values 3
- Table output (UF)

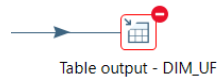


Ao fazer um **Preview** na **DIM_UF**, você verá muita repetição de UFs.

Rode os seguintes scripts no MySql:

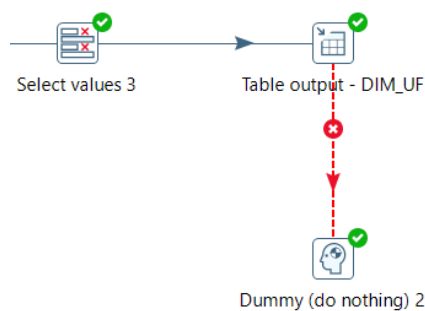
- Vamos limpar a tabela:
truncate table tp.uf;
- Vamos criar uma primary key para não termos repetição de uf:
alter table tp.uf
add primary key (sg_uf_programa);

Rode a transformação e você verá o erro de integridade referencial.

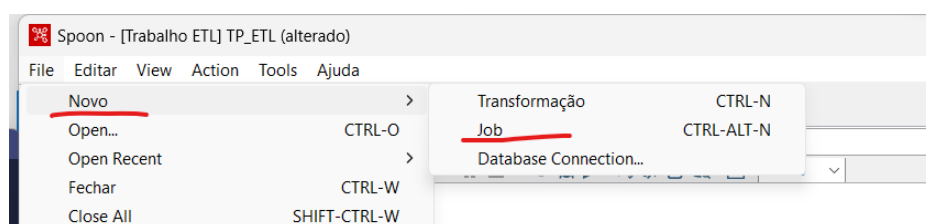


```
Execution Results
Logging Execution History Step Metrics Performance Graph Metrics Preview data
2023/09/27 22:15:38 - Table output - DIM_UF.0 - at org.pentaho.di.trans.steps.tableoutput.TableOutput.writeToTable(TableOutput.java:291)
2023/09/27 22:15:38 - Table output - DIM_UF.0 - ... 3 more
2023/09/27 22:15:38 - Table output - DIM_UF.0 - Caused by: com.mysql.jdbc.exceptions.jdbc4.MySQLIntegrityConstraintViolationException: Duplicate entry 'RJ' for key 'uf.PRIMARY'
2023/09/27 22:15:38 - Table output - DIM_UF.0 - at java.base/jdk.internal.reflect.DirectConstructorHandleAccessor.newInstance(DirectConstructorHandleAccessor.java:62)
2023/09/27 22:15:38 - Table output - DIM_UF.0 - at java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:502)
2023/09/27 22:15:38 - Table output - DIM_UF.0 - at java.base/java.lang.reflect.Constructor.newInstance(Constructor.java:486)
2023/09/27 22:15:38 - Table output - DIM_UF.0 - at com.mysql.jdbc.Util.handleNewInstance(Util.java:403)
```

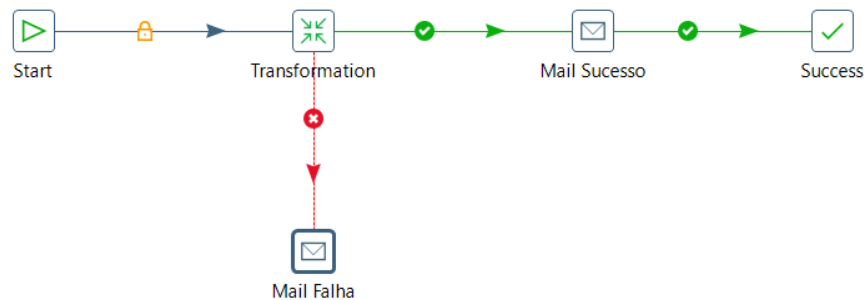
Insira um Step **Dummy** e veja o que acontece (Preview).



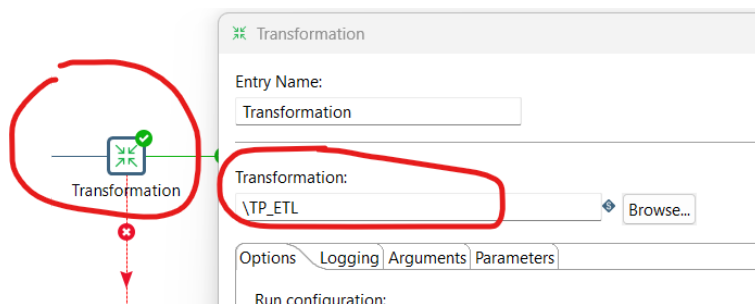
Agora vamos fazer a **orquestração** da transformação e para isso vamos criar um arquivo de **Job** conforme imagem abaixo.



Reproduza o Job, conforme Steps da imagem abaixo.



No Step **Transformation**, linkar a transformação trabalhada.

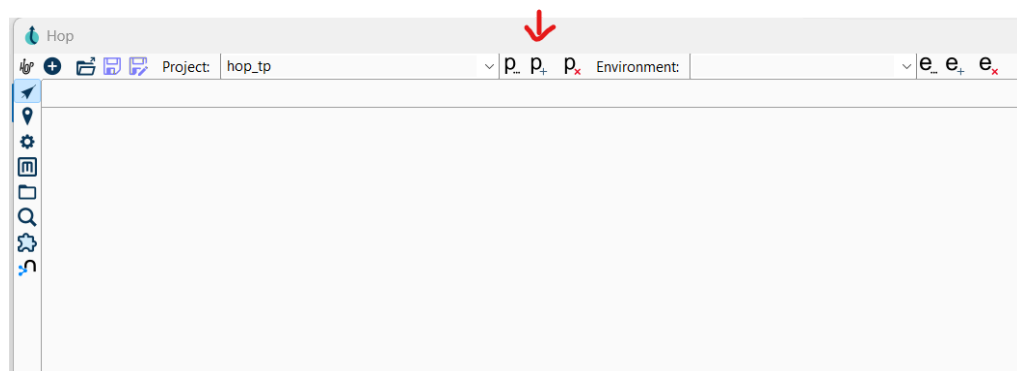


Rodar a transformação pelo Job.

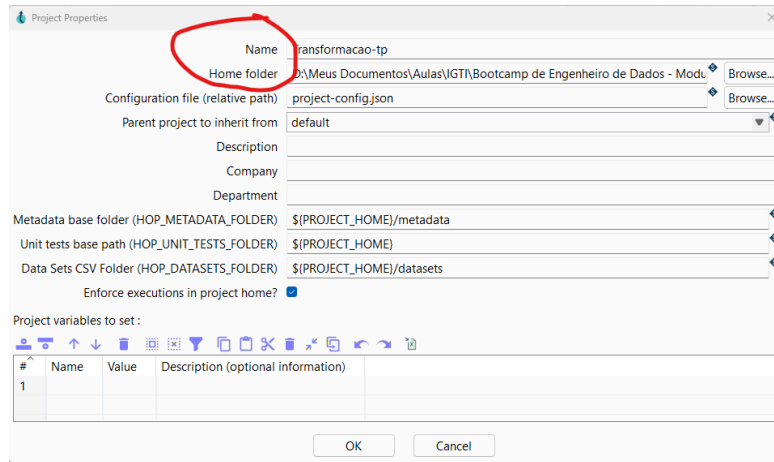
2) Prática Apache HOP

Para iniciar, vamos converter o trabalho prático executado no Pentaho PDI (Spoon.bat) para o Apache Hop.

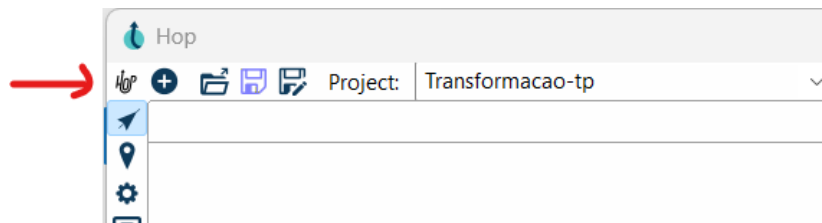
Crie um novo projeto.



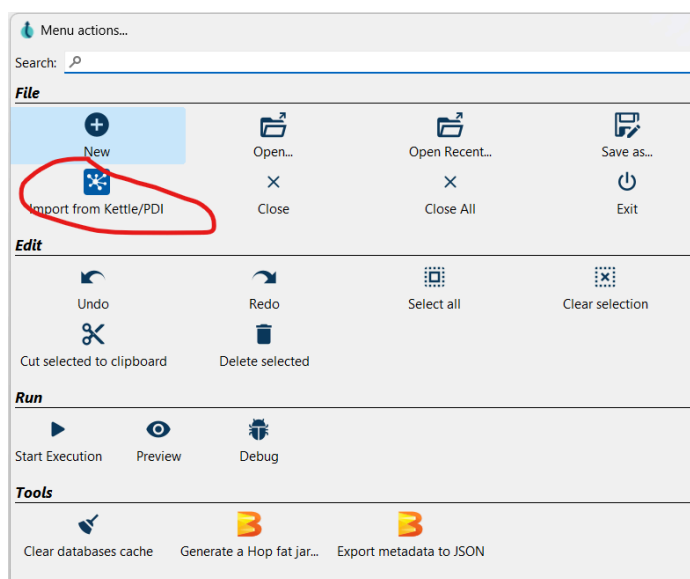
Preencha o nome e o diretório onde será armazenado o projeto.



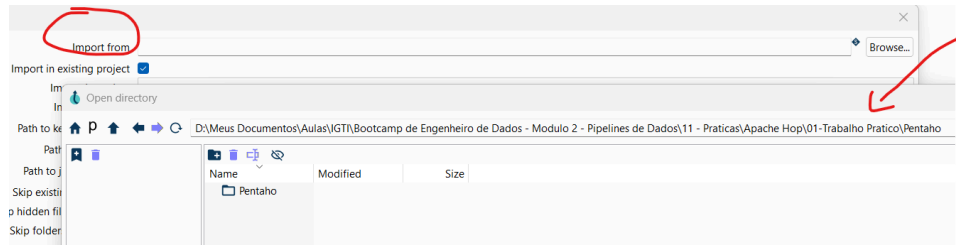
Clique no ícone Hop.



Selecione “Import from Kettle/PDI”.



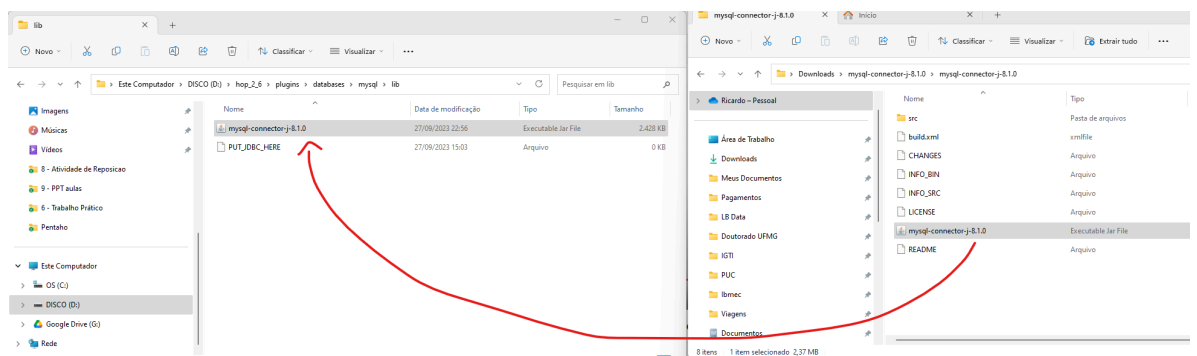
Informe o diretório onde estão os arquivos ktr (transformação) e kjb (job) do Pentaho PDI.



<https://dev.mysql.com/get/Downloads/Connector-J/mysql-connector-j-8.1.0.zip>

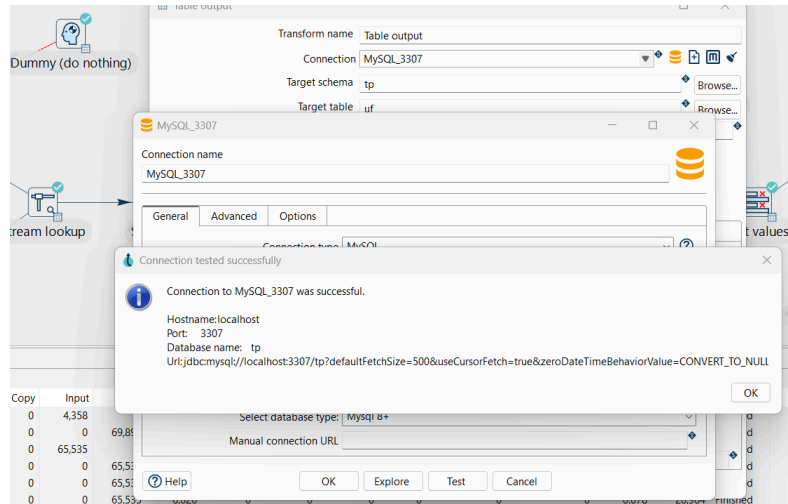
Abra o arquivo **mysql-connector-j-8.1.0.zip**.

Selecione o arquivo **mysql-connector-j-8.1.0.jar** e copie para dentro do Apache Hop, ...\\plugins\\databases\\mysql\\lib.

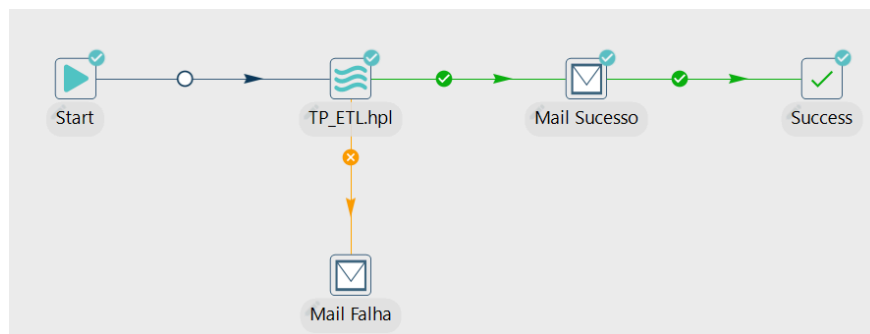


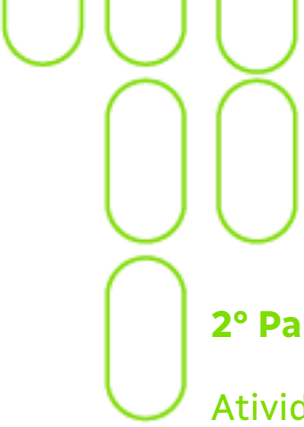
Feche o Apache Hop e abra novamente testando a conexão com o MySQL.

Se possível, crie uma conexão nova.



Agora já é possível rodar o Job no Apache Hop.





2º Parte - Opcional - Apache Airflow

Atividades

1. Instalação

Primeiro, você precisa instalar / subir uma instância do Apache Airflow. Você tem opção de usar nuvem ou instalar localmente:

- Criação de Instância do Apache Airflow no Google Cloud.
- Instalação Local.
- Instalação Local em Container – Docker.

Use um destes guias e passe para a criação de DAGs no Airflow.

a. Criação de Instância do Apache Airflow no Google Cloud

Esta é uma forma mais tranquila de se ter o Apache Airflow.

Se você ainda não tem, precisa uma conta no Google Cloud e receber créditos que permitirão a prática. O Google Cloud oferece US\$ 300 para teste gratuito. Conheça o programa de gratuidade do [Google Cloud](#). O teste gratuito se aplica à primeira conta de faturamento criada.

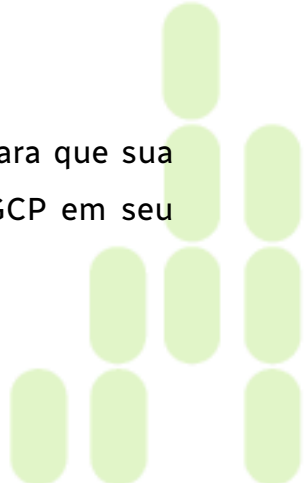
O Google Cloud é uma plataforma de computação em nuvem oferecida pela Google. Ela fornece uma ampla gama de serviços de computação, armazenamento, análise de dados, aprendizado de máquina, redes e outros recursos para ajudar indivíduos e empresas a executarem suas operações de TI de forma eficiente e escalável na nuvem.

i. Crie um Projeto no Google Cloud

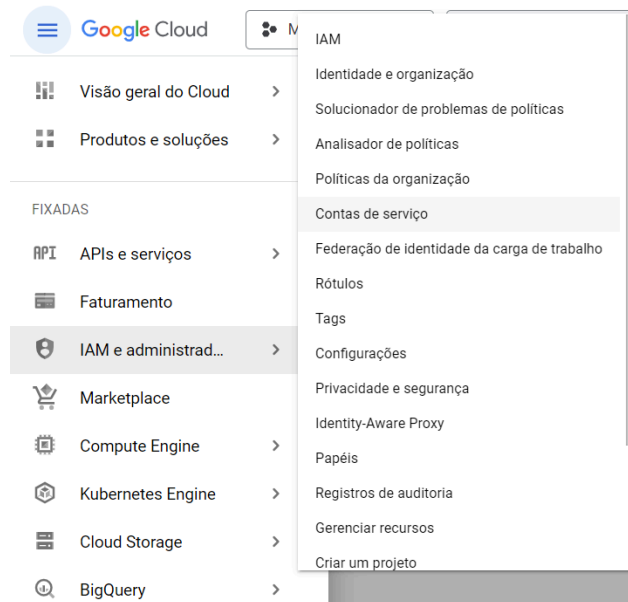
Se você ainda não possui um projeto no Google Cloud, precisa criar um.

ii. Create a Service Account

No projeto criado, você precisará criar uma Conta de Serviço para que sua instância do Airflow possa acessar os diferentes serviços do GCP em seu projeto.



Primeiro, vá para IAM & Admin e depois Contas de serviço.



Em seguida, digite o nome da conta de serviço e clique em Criar.

← Criar conta de serviço

1 Detalhes da conta de serviço

Nome da conta de serviço
Airflow

Nome de exibição para esta conta de serviço

ID da conta de serviço *
airflow

Endereço de e-mail: airflow@titanium-campus-395715.iam.gserviceaccount.com

Descrição da conta de serviço
Instancia do Airflow

Descreva como a conta de serviço será usada

criar e continuar

2 Conceda a essa conta de serviço acesso ao projeto (opcional)

3 Conceda aos usuários acesso a essa conta de serviço (opcional)

concluir CANCELAR

Contas de serviço para o projeto "My First Project"

Uma conta de serviço representa uma identidade de serviço do Google Cloud, como o código que é executado nas VMs do Compute Engine, aplicativos do App Engine ou sistemas executados fora do Google. [Saiba mais sobre contas de serviço.](#)

Políticas da organização podem ser usadas para proteger contas de serviço e bloquear recursos arriscados de conta de serviço, como IAM Grants automático, criação/upload de chaves ou a criação de contas de serviço inteiramente. [Saiba mais sobre políticas da organização da conta de serviço.](#)

Filtro Insira o nome ou o valor da propriedade							
☐	E-mail	Status	Nome ↑	Descrição	Código da chave	Data de criação da	Ações
☐	airflow@titanium-campus-395715.iam.gserviceaccount.com	Ativado	Airflow	Instancia do Airflow	Nenhuma chave		⋮

Criar um ambiente do Google Composer

Um ambiente do Cloud Composer é uma instalação autônoma do Apache Airflow implantada em um cluster gerenciado do Google Kubernetes Engine. É possível criar um ou mais ambientes em um único projeto do Google Cloud.

Criar um ambiente do Cloud Composer

O Cloud Composer é um serviço totalmente gerenciado de orquestração de fluxos de trabalho criado no Apache Airflow. Com a automação do Cloud Composer, é possível criar ambientes do Airflow e usar as ferramentas nativas dele, como a interface da Web avançada do Airflow e as ferramentas de linha de comando. Assim, você se concentra nos fluxos de trabalho, e não na infraestrutura.

Por padrão, os ambientes do Cloud Composer usam a conta de serviço padrão do Compute Engine gerenciada pelo Google. Recomendamos que você crie uma conta de serviço gerenciada pelo usuário que tenha um papel específico para o Cloud Composer e use-a nos seus ambientes.

O tempo aproximado para criar um ambiente é de 25 minutos.

Durante a criação do ambiente, você especifica uma conta de serviço. Os nós no cluster do ambiente são executados como esta conta.

Antes de começar, ative a API Cloud Composer.

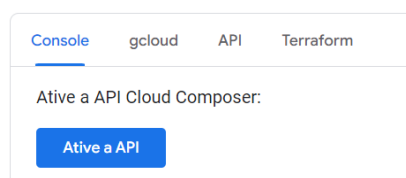
Ativar a API Cloud Composer.

Vá em:

<https://cloud.google.com/composer/docs/how-to/managing/enabling-service?hl=pt-br>

Localize a parte de ativação de API do Cloud Composer.

Ativar a API Cloud Composer



Após você ativar as apis, se tentar novamente vai receber a seguinte mensagem:

As APIs necessárias estão ativadas:

- Cloud Build API
- Cloud Functions API
- Cloud Composer API

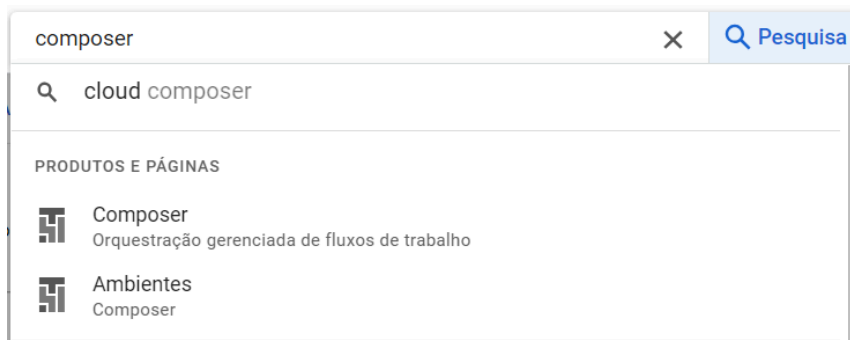
Você pode configurar o ambiente de Cloud Shell, se desejar e continuar a configuração pelo Cloud Shell. Nós iremos continuar pela interface gráfica, mas fácil e intuitiva.

Procure a opção pela ajuda: Abrir o Cloud Shell.

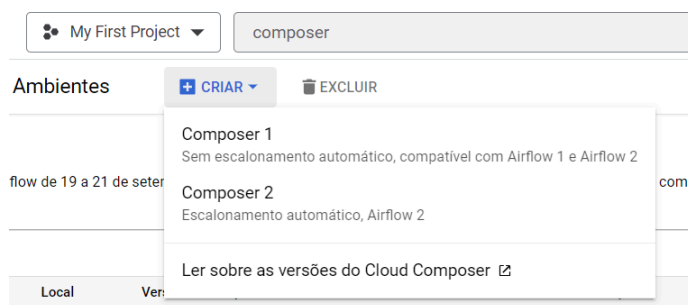
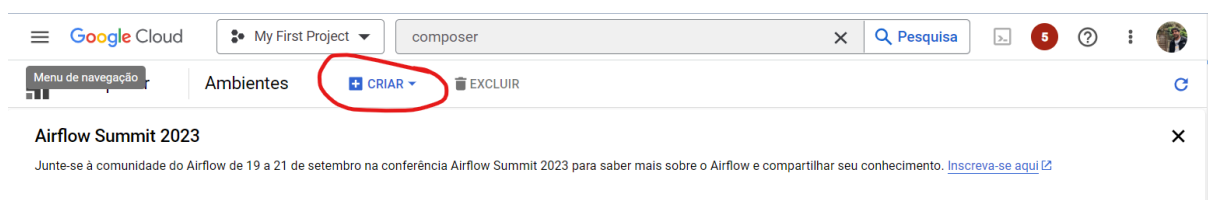
1. Clique em  **Ativar o Cloud Shell** para abrir o Cloud Shell. Também é possível abrir o Cloud Shell clicando no botão:

 **Abrir o Cloud Shell**

Procure pelo campo de ajuda do Google Cloud “Composer” e abra a orquestração “Composer”.



Crie um “Composer1” compatível com Airflow 1 e Airflow 2



Preencha os campos.

 My First Project

 **Composer** ← Criar ambiente

Nome *

testeairflow

Local *

us-east1

Região do Google Cloud Platform em que o ambiente será criado.

Versão da imagem *

composer-1.20.12-airflow-2.4.3

A versão da imagem a ser usada no ambiente criado. O valor da versão da imagem inclui a versão do Composer e do Airflow. É preciso especificar um local antes selecionar a versão da imagem.

Configuração de nós

As informações de configuração para os nós do Google Kubernetes Engine em execução no software Airflow.

Contagem de nós *

3

O número de nós no cluster do Google Kubernetes Engine que será usado para executar esse ambiente.

Zona

us-east1-b

Zona do Google Cloud Platform em que os nós do cluster serão implantados. Especifique um local antes de selecionar a zona. [Saiba mais](#).

Tipo de máquina

n1-standard-1

O tipo de máquina do Google Compute Engine usado nas instâncias do cluster. Se não for especificado, o tipo de máquina terá o padrão "n1-standard-1". É preciso especificar um local antes de selecionar o tipo de máquina. [Saiba mais](#).

Tamanho do disco (GB)

30

O tamanho do disco em GB usado para VMs de nós. O mínimo é 20 GB. Caso esse campo não seja especificado, o padrão será 100 GB. Não é possível atualizar esse valor.

Escopos do OAuth

O conjunto de escopos da API do Google a serem disponibilizados em todas as VMs do nó. Caso o campo esteja vazio, o padrão será <https://www.googleapis.com/auth/cloud-platform>. Não é possível atualizar esse valor. [Saiba mais](#).

Conta de serviço

A conta de serviço do Google Cloud Platform a ser usada pelas VMs do nó. Caso o campo não seja especificado, será usado o padrão da conta de serviço do Compute Engine. Não é possível atualizar esse valor.

Tags

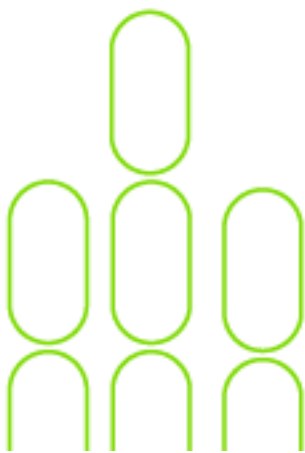
A lista de tags de instância aplicadas a todas as VMs de nós. As tags são usadas para identificar fontes ou destinos válidos para firewalls de rede. Cada tag com a lista precisa estar de acordo com o RFC1035. Não é possível atualizar esse valor.

Número de programadores *

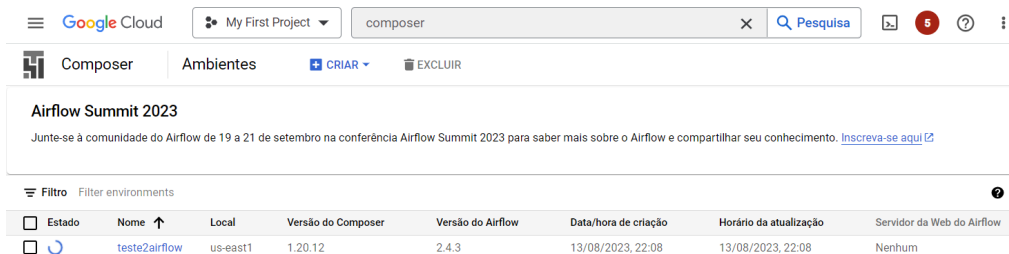
1

Seu ambiente pode executar mais de um programador do Airflow ao mesmo tempo.

✓ REDES, ALTERAÇÃO DE CONFIGURAÇÕES DE FLUXO DE AR E RECURSOS ADICIONAIS.



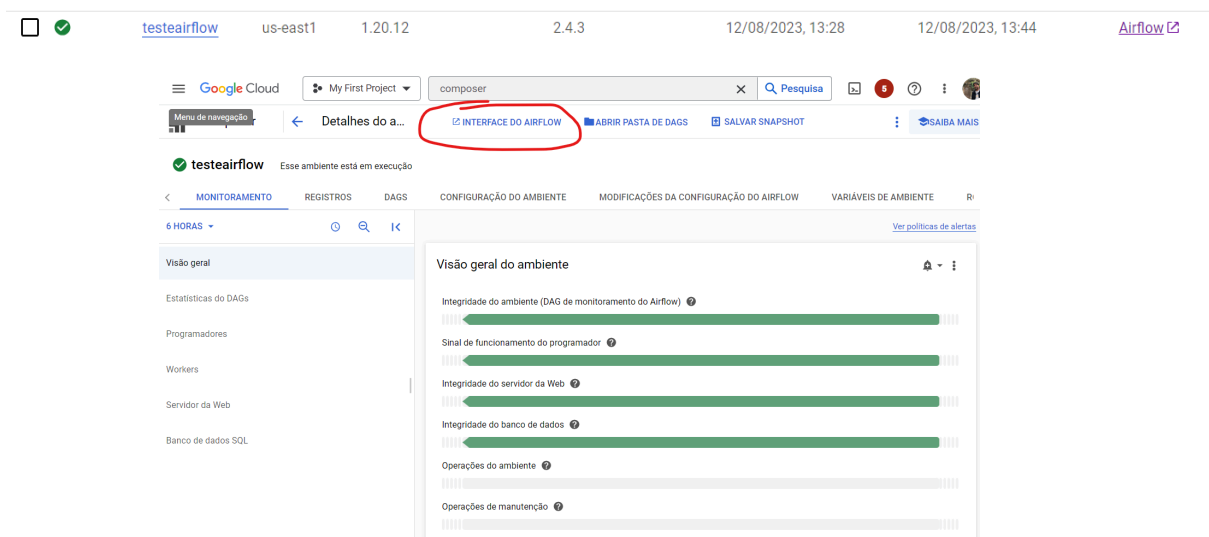
Você pode marcar a opção de usar a API Beta.



The screenshot shows the Google Cloud Composer console. At the top, there's a header with 'Google Cloud', 'My First Project', and a search bar. Below, there's a section for 'Airflow Summit 2023'. The main table lists environments. The first environment is 'testeairflow' with status 'Running' (green checkmark), location 'us-east1', Composer version '1.20.12', Airflow version '2.4.3', creation time '13/08/2023, 22:08', last update '13/08/2023, 22:08', and 'Nenhum' for the web server.

Estado	Nome	Local	Versão do Composer	Versão do Airflow	Data/hora de criação	Horário da atualização	Servidor da Web do Airflow
☒	testeairflow	us-east1	1.20.12	2.4.3	13/08/2023, 22:08	13/08/2023, 22:08	Nenhum

Depois é só clicar no link do Composer criado para acessar o ambiente.



The screenshot shows the Airflow web interface. The top navigation bar includes 'Airflow', 'DAGs', 'Datasets', 'Browse', 'Admin', and 'Docs'. The main header shows 'testairflow' and 'Active' status. Below, there's a table with columns: DAG, Owner, Runs, Schedule, Last Run, Next Run, Recent Tasks, Actions, and Links. The first row shows 'airflow_monitoring' with a green status circle and a link to 'airflow'.

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions	Links
airflow_monitoring	airflow	●	2023-08-14, 01:10:00	2023-08-14, 01:20:00				

Clique em Interface do Airflow e você terá o Airflow acessível.

Alerta: você recebe 300 dólares em créditos, que podem ser facilmente consumidos ao longo do tempo.

Por isso, ao terminar o trabalho, desabilite a instância que contém o Apache Airflow para evitar cobranças futuras.



b. Instalação Local

A instalação do Apache Airflow localmente envolve alguns passos. O processo básico de instalação se dá pelo uso do pip, que é o gerenciador de pacotes Python. Certifique-se de que você possui o Python instalado em sua máquina antes de começar. Aqui estão os passos:

i. Crie um Ambiente Virtual (Opcional, mas recomendado):

Criar um ambiente virtual é uma prática recomendada para isolar as dependências do projeto. Para criar um ambiente virtual, abra o terminal e navegue até o diretório onde deseja criar o ambiente. Em seguida, execute o seguinte comando:

```
python -m venv my_airflow_env
```

```
python -m venv my_airflow_env
```

Isso criará um ambiente virtual chamado my_airflow_env. Para ativá-lo, use:

No Windows:

```
my_airflow_env\Scripts\activate
```

```
my_airflow_env\Scripts\activate
```

No macOS e Linux:

```
source my_airflow_env/bin/activate
```

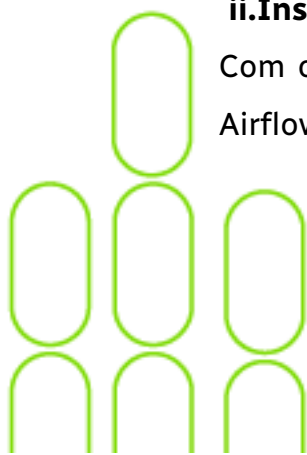
```
source my_airflow_env/bin/activate
```

ii. Instale o Apache Airflow:

Com o ambiente virtual ativado, use o comando pip para instalar o Apache Airflow:

```
pip install apache-airflow
```

```
pip install apache-airflow
```





iii. Inicialize o Banco de Dados:

O Airflow usa um banco de dados para armazenar metadados sobre DAGs, tarefas, execuções etc. Você precisará inicializar o banco de dados:

```
airflow db init
```

```
airflow db init
```

iv. Inicie o Web Server do Airflow:

Inicie o servidor da web Airflow para acessar o painel de controle:

```
airflow webserver --port 8080
```

```
airflow webserver --port 8080
```

v. Inicie o Scheduler:

Abra outro terminal e inicie o scheduler, responsável por agendar e executar tarefas:

```
airflow scheduler
```

```
airflow scheduler
```

vi. Acesse o Painel Web:

Abra um navegador e acesse <http://localhost:8080>. Isso o levará ao painel de controle do Airflow, onde você pode criar e monitorar suas DAGs.

Lembre-se de que esta é uma configuração básica para uso local.





c. Instalação Local em Container - Docker

A instalação do Apache Airflow através do Docker é uma abordagem popular, pois facilita a configuração e execução do Airflow em diferentes ambientes sem a necessidade de configurar dependências complexas manualmente. Aqui estão os principais passos para instalar o Apache Airflow usando o Docker.

i. Criar uma instância do WSL 2 no Windows.

Verificar Requisitos:

Verifique se você está executando pelo menos o Windows 10 versão 1903 com compilação do sistema operacional 18362 ou superior.

Certifique-se de que a virtualização esteja habilitada na BIOS do seu computador.

Ativar o WSL:

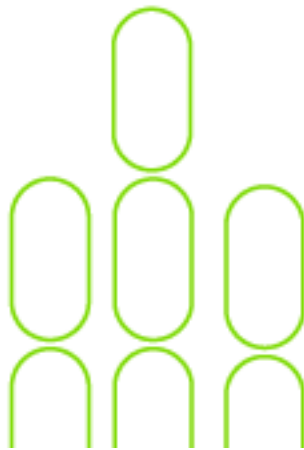
Abra o "Painel de Controle" e acesse "Programas" > "Ativar ou desativar recursos do Windows".

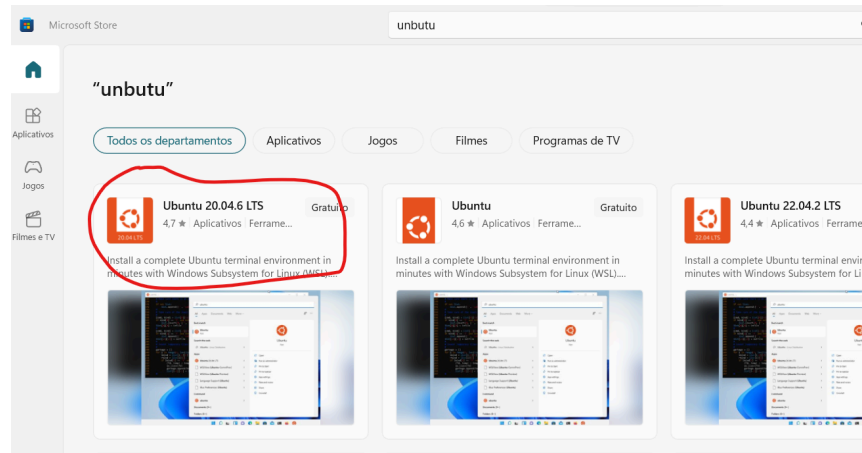
Marque a opção "Plataforma de Máquina Virtual" e clique em "OK". Reinicie o computador quando solicitado.

Instalar uma Distribuição Linux via Microsoft Store:

Abra a "Microsoft Store" e procure por uma distribuição Linux compatível (por exemplo, Ubuntu).

Escolha uma distribuição, clique em "Obter" e siga as instruções para instalá-la.





Configurar a Distribuição Linux:

Após a instalação, execute a distribuição Linux recém-instalada no menu Iniciar.

A primeira vez que você executa a distribuição, será solicitado que você configure um nome de usuário e senha.

Atualizar para WSL 2:

Abra o **PowerShell** como administrador.

Execute o comando: `wsl --set-version <Nome da Distribuição> 2`, substituindo <Nome da Distribuição> pelo nome da distribuição Linux que você instalou.

Aguarde o processo de atualização. As novas distribuições já estão no padrão da wsl 2.

Definir WSL 2 como Padrão:

Execute o comando no PowerShell: `wsl --set-default-version 2`.

Instalar o Kernel Linux:

Baixe e instale a atualização do kernel do Linux para o WSL 2. Veja as orientações no link: <https://aka.ms/wsl2kernel>.

Basicamente execute no terminal o comando: `ws1.exe --update`



```
C:\Users\ricar>wsl.exe --update
Instalando: Subistema do Windows para Linux
[=====52,0%]
```

Verificar a Versão do WSL:

Execute o comando no PowerShell: `wsl --list --verbose` para ver as distribuições instaladas e suas versões. A coluna "Versão" deve ser "2".

```
C:\Users\ricar>wsl --list --verbose
NAME                STATE      VERSION
* Ubuntu             Stopped    2
docker-desktop-data  Stopped    2
docker-desktop       Stopped    2
```

Utilizar a Distribuição Linux: agora você pode iniciar a distribuição Linux via menu Iniciar, PowerShell ou Terminal, digitando o comando `wsl` ou o nome da distribuição (por exemplo, ubuntu).

ii.Instalação do Docker

Se você ainda não tem o Docker instalado, você precisa instalá-lo primeiro. Você pode baixar o Docker Desktop para Windows ou macOS, ou instalar o Docker Engine em sistemas Linux. Acesse o site oficial do Docker para obter instruções detalhadas de instalação. No caso do Windows, o Docker funciona na versão home, porém é necessário utilizar o WSL 2 (Subsistema Windows para Linux 2) para criar um ambiente de execução para contêineres. O Docker Desktop pode usar o WSL 2. Isso não requer uma máquina virtual tradicional, mas WSL 2 é uma tecnologia que fornece um ambiente semelhante a uma VM para a execução de contêineres.

iii.Preparação dos Arquivos de Configuração

Antes de iniciar um contêiner Docker com o Apache Airflow, é recomendável preparar alguns arquivos de configuração. Você pode criar um diretório para armazenar esses arquivos e personalizá-los conforme necessário.



iv.Arquivo docker-compose.yaml

O arquivo docker-compose.yaml (ou docker-compose.yml) é um arquivo de configuração que define como vários serviços Docker serão implantados juntos. Ele é usado com a ferramenta Docker Compose para orquestrar a execução de múltiplos containers de maneira simples. Nesse arquivo, você especifica todos os serviços que compõem a sua aplicação (como bancos de dados, servidores web, workers, etc.), suas dependências, volumes, redes, variáveis de ambiente, entre outras configurações.

Aqui está uma estrutura básica do que pode ser encontrado em um docker-compose.yaml:

yaml

version: "3" # Especifica a versão do Docker Compose

services: # Define os serviços (containers) que serão iniciados

app: # Nome do serviço

image: node:14 # Imagem Docker usada para este serviço

volumes:

- ./app:/usr/src/app # Mapeamento de volumes (diretório local para o container)

environment:

NODE_ENV: development # Variáveis de ambiente

ports:

- "3000:3000" # Mapeamento de portas (host:container)

db: # Outro serviço, por exemplo, um banco de dados

image: postgres:latest

environment:

POSTGRES_USER: user

POSTGRES_PASSWORD: password

volumes:



```
- db_data:/var/lib/postgresql/data
```

```
ports:
```

```
- "5432:5432"
```

```
volumes:
```

```
db_data: # Define volumes para persistência de dados
```

Componentes principais:

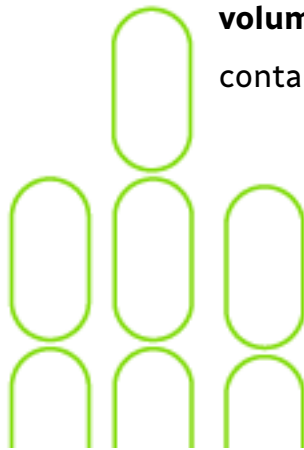
version: Especifica a versão da sintaxe do Docker Compose que está sendo usada.

services: Define os containers que serão executados. Cada serviço pode ser configurado com diferentes opções, como:

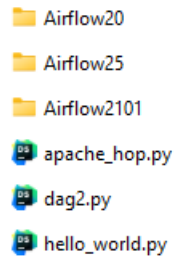
image: A imagem do Docker que será usada para criar o container.

- o **build:** Caminho para o Dockerfile, caso o serviço precise ser construído localmente.
- o **ports: Mapeamento de portas (host) para expor o container ao ambiente externo.**
- o **volumes:** Monta volumes entre o sistema de arquivos do host e o container.
- o **environment:** Define variáveis de ambiente usadas pelo container.
- o **depends_on:** Define a ordem de inicialização entre os serviços.

volumes: Permite a criação de volumes que podem ser compartilhados entre containers ou persistentes após o encerramento do container.



Use o arquivo zip anexo ao trabalho com a seguinte estrutura de diretórios:



Sugiro subir a versão completa do Airflow, que está no diretório Airflow2101 (Airflow versão 2.10.1). Os outros diretórios têm as versões mais antigas do Airflow (2.0.0 e 2.5.1).

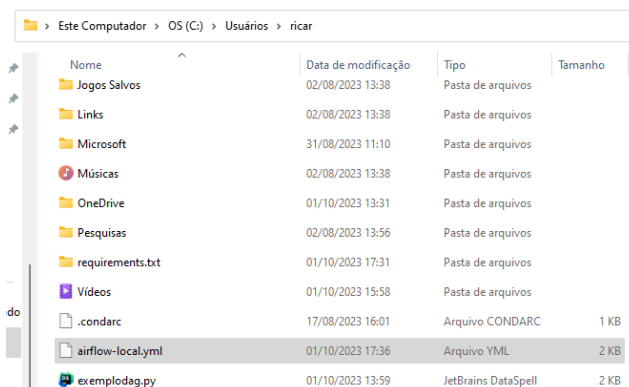
Dentro do diretório Airflow2101 tem um arquivo *yml* *airflow-local* disponibilizado. Você também pode baixar, conforme abaixo.

- Versão 2.5.1: [airflow-local.yml](#)
- Versão 2.10.1: [airflow-local.yml](#)

v.Iniciar o Apache Airflow:

Abra um terminal e navegue para o diretório onde você criará a sua estrutura de diretórios do Airflow.

Por exemplo, coloque o arquivo yml do compose no diretório de usuário.





Abra o terminal e vá para o diretório de usuário onde estão os arquivos airflow-local.yml e o exemplodag.py

Execute o seguinte comando:

```
docker-compose -f airflow-local.yml up -d
```

Após a criação dos containers, vá no Docker Desktop e confira se estão rodando.

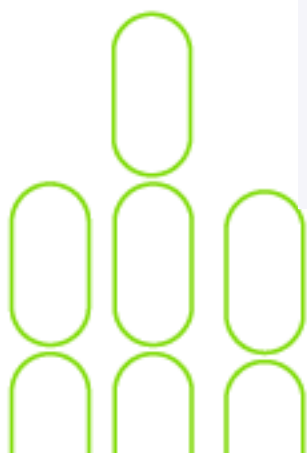
```
D:\DockerCompose\Airflow\Airflow2101>docker-compose -f airflow-local.yml up -d
time="2024-10-21T10:48:15-03:00" level=warning msg="The \"AIRFLOW_UID\" variable is not set. Defaulting to a blank string."
time="2024-10-21T10:48:15-03:00" level=warning msg="The \"AIRFLOW_UID\" variable is not set. Defaulting to a blank string."
[+] Running 49/7
  ✓ airflow-init Pulled
  ✓ postgres 14 layers [#####] 0B/0B Pulled 66.8s
  ✓ airflow-worker Pulled 22.7s
  ✓ airflow-triggerer 22 layers [#####] 0B/0B Pulled 66.8s
  ✓ redis 6 layers [#####] 0B/0B Pulled 66.8s
  ✓ airflow-webserver Pulled 19.4s
  ✓ airflow-scheduler Pulled 66.8s
[+] Building 0.0s (0/0) docker:default
[+] Running 9/9
  ✓ Network airflow2101_default Created 0.0s
  ✓ Volume "airflow2101_postgres-db-volume" Created 0.0s
  ✓ Container airflow2101-redis-1 Healthy 0.2s
  ✓ Container airflow2101-postgres-1 Healthy 0.2s
  ✓ Container airflow2101-airflow-init-1 Exited 0.1s
  ✓ Container airflow2101-airflow-worker-1 Started 0.1s
  ✓ Container airflow2101-airflow-scheduler-1 Started 0.1s
  ✓ Container airflow2101-airflow-webserver-1 Started 0.1s
  ✓ Container airflow2101-airflow-triggerer-1 Started 0.1s
D:\DockerCompose\Airflow\Airflow2101>
```

Você também pode checar pelo terminal:

```
docker container ps -a
```

Você também pode checar pelo Docker Desktop:

airflow2101		Running (6/7)	3.06%	1 minute ago
postgres-1	postgres:13	Running	0.72%	1 minute ago
redis-1	redis:7.2-bookworm	Running	0.12%	1 minute ago
airflow-init-1	apache/airflow:2.10.1	Exited	0%	1 minute ago
airflow-scheduler-1	apache/airflow:2.10.1	Running	1.44%	1 minute ago
airflow-webserver-1	apache/airflow:2.10.1	Running	0.1% 8081:8080	1 minute ago
airflow-worker-1	apache/airflow:2.10.1	Running	0.05%	1 minute ago
airflow-triggerer-1	apache/airflow:2.10.1	Running	0.63%	1 minute ago



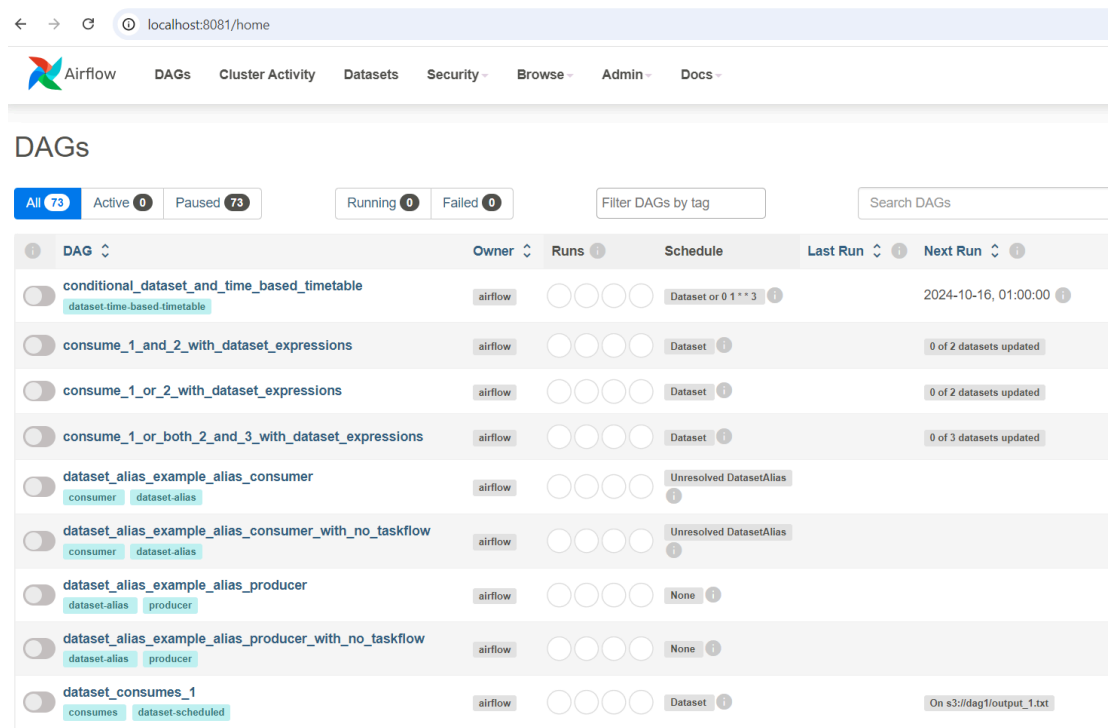
vi. Acesso ao Painel Web:

Depois que o contêiner estiver em execução, você pode acessar o painel de controle do Airflow no seu navegador. Na imagem acima, o endereço de acesso ao Airflow é:

<http://localhost:8081>

user: airflow

password: airflow



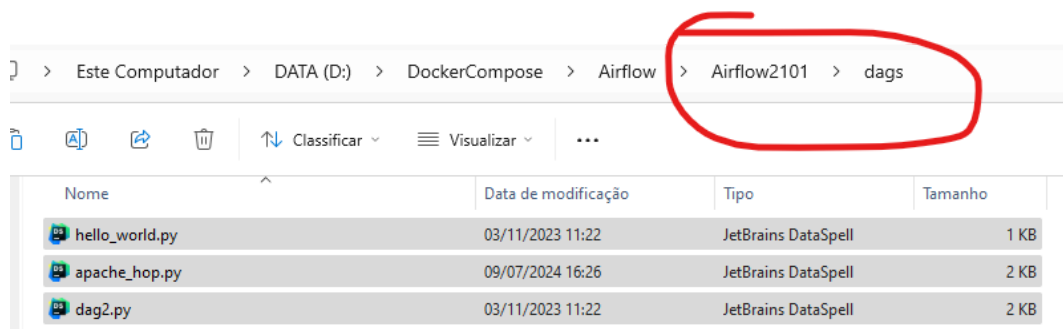
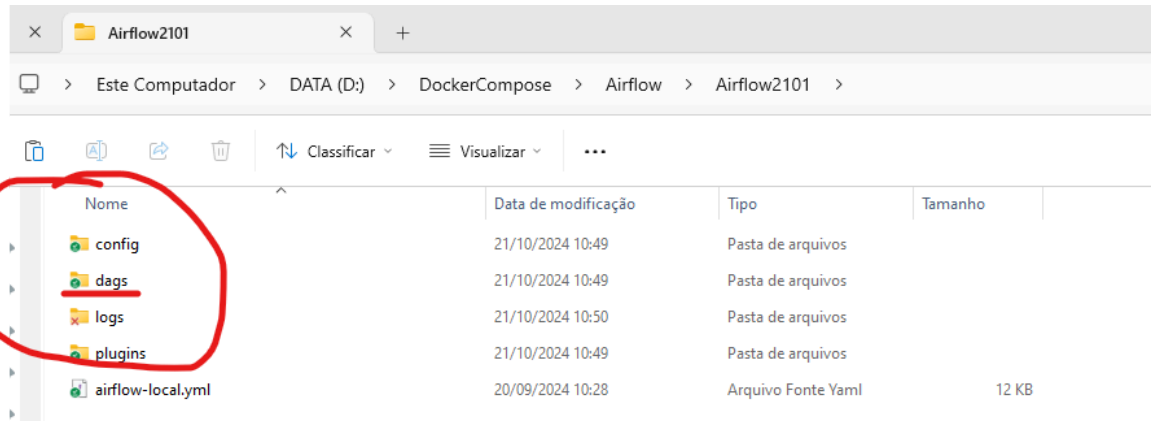
The screenshot shows the Apache Airflow web interface at localhost:8081/home. The top navigation bar includes links for Airflow, DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The main section is titled 'DAGs' and features a filter bar with buttons for 'All' (73), 'Active' (0), and 'Paused' (73). There are also buttons for 'Running' (0) and 'Failed' (0), a 'Filter DAGs by tag' input, and a 'Search DAGs' search bar. Below the filter bar is a table listing various DAGs. Each row includes a toggle switch, the DAG name, its owner (all are 'airflow'), a visual representation of its runs, its schedule, and its last and next run times. Some DAGs have associated dataset information.

DAG	Owner	Runs	Schedule	Last Run	Next Run
☐ conditional_dataset_and_time_based_timetable dataset.time-based-timetable	airflow	○ ○ ○ ○ ○	Dataset or 0 1 * * 3		2024-10-16, 01:00:00
☐ consume_1_and_2_with_dataset_expressions	airflow	○ ○ ○ ○ ○	Dataset		0 of 2 datasets updated
☐ consume_1_or_2_with_dataset_expressions	airflow	○ ○ ○ ○ ○	Dataset		0 of 2 datasets updated
☐ consume_1_or_both_2_and_3_with_dataset_expressions	airflow	○ ○ ○ ○ ○	Dataset		0 of 3 datasets updated
☐ dataset_alias_example_alias_consumer consumer dataset-alias	airflow	○ ○ ○ ○ ○	Unresolved DatasetAlias		
☐ dataset_alias_example_alias_consumer_with_no_taskflow consumer dataset-alias	airflow	○ ○ ○ ○ ○	Unresolved DatasetAlias		
☐ dataset_alias_example_alias_producer dataset-alias producer	airflow	○ ○ ○ ○ ○	None		
☐ dataset_alias_example_alias_producer_with_no_taskflow dataset-alias producer	airflow	○ ○ ○ ○ ○	None		
☐ dataset_consumes_1 consumes dataset-scheduled	airflow	○ ○ ○ ○ ○	Dataset		On s3://dag1/output_1.txt

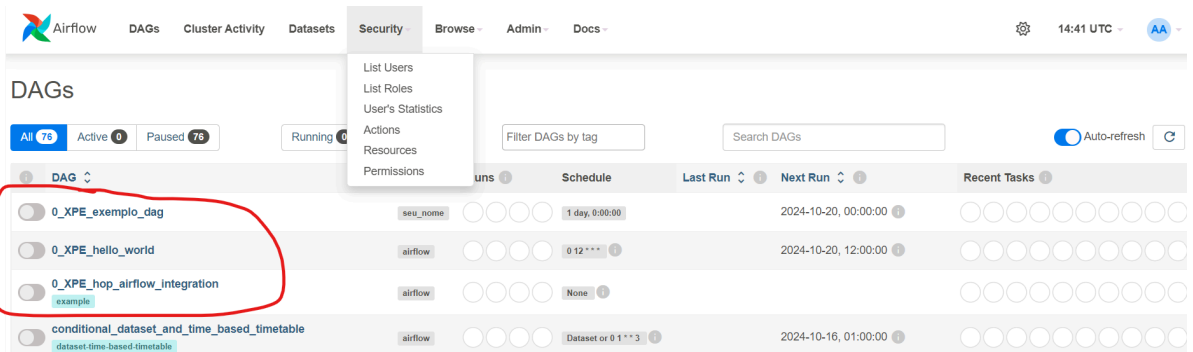
Repare que você sobe o Airflow com várias DAGs como exemplo.

vii. Para subir DAGs no Aiflow do Docker:

Basta colocar os arquivos das DAGs em Python no diretório dags que está abaixo do local onde você escolheu para seu Airflow.



O DAG leva um tempo para aparecer no Apache Airflow.
Se não aparecer, pare o conjunto de containers e os inicie novamente.



1. Vamos criar uma DAG.

a. Definindo um Fluxo de Trabalho

O Airflow permite definir fluxos de trabalho usando código Python. Cada fluxo de trabalho é conhecido como "DAG" (Directed Acyclic Graph), que consiste em tarefas e suas dependências. Cada tarefa é uma unidade de trabalho que pode ser algo como uma etapa de processamento de dados, um script Python, uma chamada de API, etc.

Aqui está um exemplo de como um simples DAG pode ser definido:

```
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from datetime import datetime

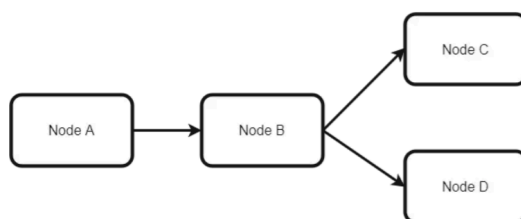
def hello_world():
    print("Hello, World!")

default_args = {
    'owner': 'you',
    'start_date': datetime(2023, 1, 1),
    'retries': 1
}

dag = DAG('my_dag', default_args=default_args, schedule_interval='@daily')

task_hello = PythonOperator(
    task_id='hello_task',
    python_callable=hello_world,
    dag=dag
)
```

Um DAG, em resumo, serve para projetar um fluxo de trabalho. Devemos pensar em dividir o fluxo de trabalho em pequenas tarefas que podem ser executadas independentemente umas das outras.





O próximo aspecto a ser compreendido é o significado de um **Nó** em um DAG. Um **Nó** nada mais é do que um **operador**. Um **operador** é uma classe que contém a lógica do que queremos alcançar no DAG. Por exemplo, se quisermos executar um script Python, teremos um **operador Python**. Se quisermos executar um comando **Bash**, temos o **operador Bash**. Existem vários operadores embutidos disponíveis para nós como parte do Airflow. A documentação sobre eles pode ser encontrada em https://airflow.apache.org/docs/apache-airflow/stable/_api/airflow/operators/index.html.

Quando um operador específico é acionado, ele se torna uma tarefa e é executado como parte da execução geral do DAG.

b. Criando um DAG Hello World

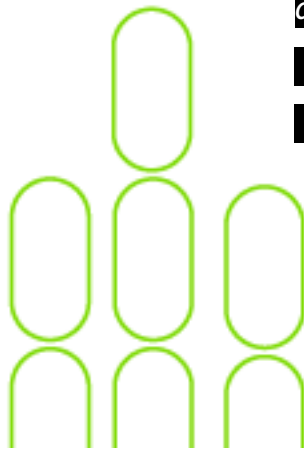
Considerando que o Airflow já esteja configurado, criaremos nosso primeiro DAG hello world. Tudo o que ele fará é imprimir uma mensagem no log.

Abaixo está o código para o DAG.

```
from datetime import datetime
from airflow import DAG
from airflow.operators.dummy_operator import DummyOperator
from airflow.operators.python_operator import PythonOperator

def print_hello():
    return 'Hello world from first Airflow DAG!'

dag = DAG('hello_world', description='Hello World DAG',
          schedule_interval='0 12 * * *',
          start_date=datetime(2023, 8, 13), catchup=False)
```



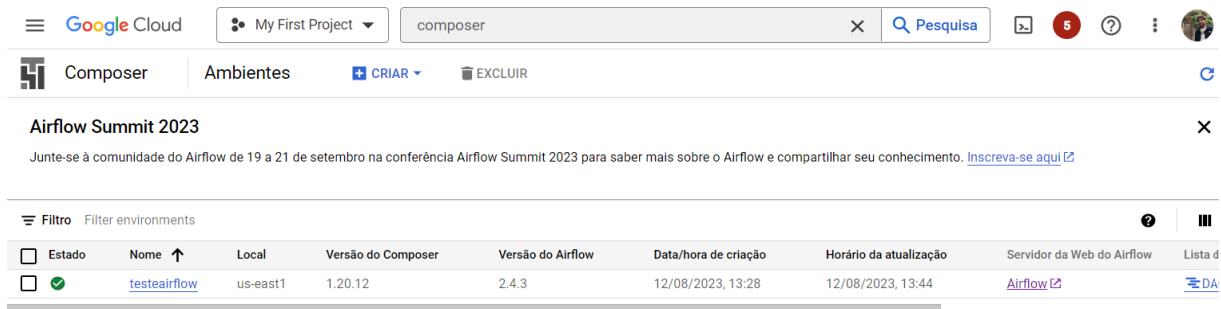
```
hello_operator = PythonOperator(task_id='hello_task',  
python_callable=print_hello, dag=dag)
```

```
hello_operator
```

Vamos dar o nome de **hello_world.py**.

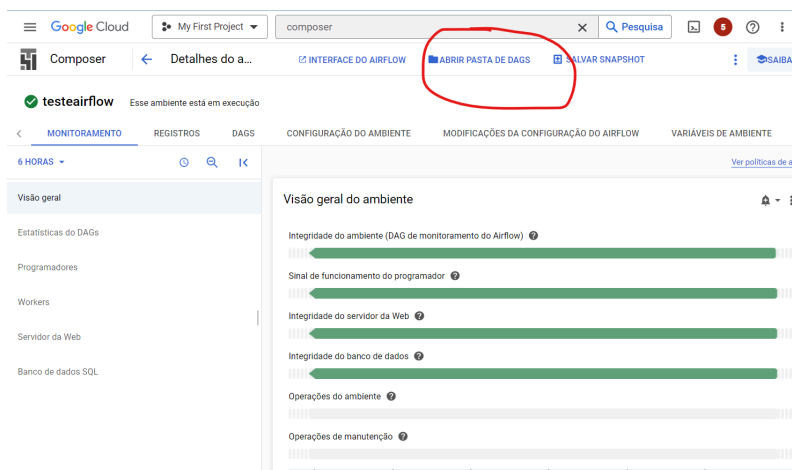
c. Subindo a DAG Hello World no Ambiente de Nuvem – Google Composer

Volte no ambiente do Composer.



Estado	Nome	Local	Versão do Composer	Versão do Airflow	Data/hora de criação	Horário da atualização	Servidor da Web do Airflow	Lista d
☒	testeairflow	us-east1	1.20.12	2.4.3	12/08/2023, 13:28	12/08/2023, 13:44	Airflow	DA

Clique no link do nome e depois em “Abrir Pasta de DAGS”.



Visão geral do ambiente

- Integridade do ambiente (DAG de monitoramento do Airflow)
- Sinal de funcionamento do programador
- Integridade do servidor da Web
- Integridade do banco de dados
- Operações do ambiente
- Operações de manutenção

Faça upload do arquivo **hello_world.py** para o bucket.

Após o upload, o DAG pode levar uns minutos para aparecer no Airflow.

Se estiver no Airflow local ou no Docker, coloque o arquivo Python do DAG na pasta “dag”.



d. Subindo a DAG Hello World no Ambiente Local, seja Docker ou local

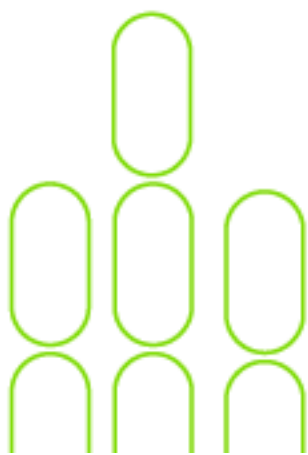
Vamos entender o que fizemos no arquivo:

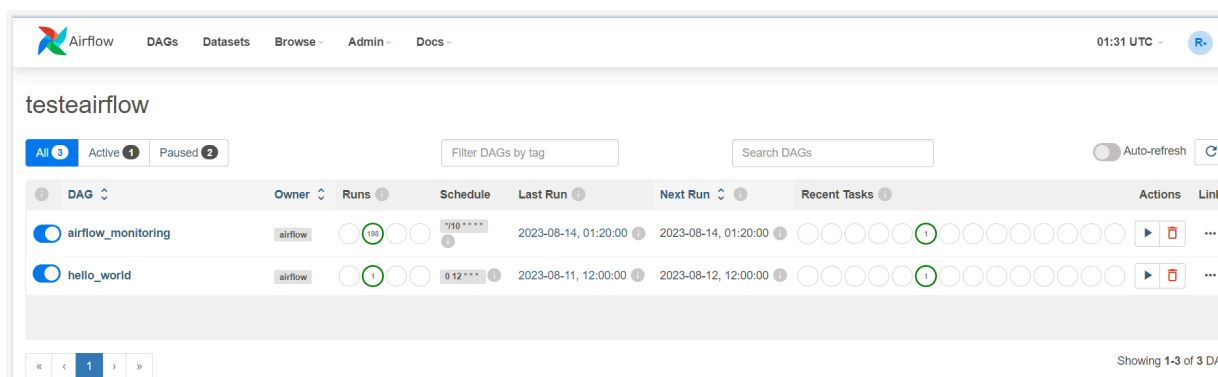
- 1) Nas primeiras linhas, estamos simplesmente importando alguns pacotes do Airflow.
- 2) Em seguida, definimos uma função que imprime a mensagem de saudação.
- 3) Depois disso, declaramos o DAG. São necessários argumentos como nome, descrição, `schedule_interval`, `start_date` e `catchup`. A configuração de `catchup` como `false` impede que o Airflow faça com que as execuções do DAG alcancem a data atual.
- 4) Em seguida, definimos o operador e o chamamos de `hello_operator`. Em essência, isso usa o `PythonOperator` embutido para chamar nossa função `print_hello`. Nós também fornecemos uma `task_id` para este operador.
- 5) A última instrução especifica a ordem dos operadores. Neste caso, temos apenas um operador.

a. Executando o DAG

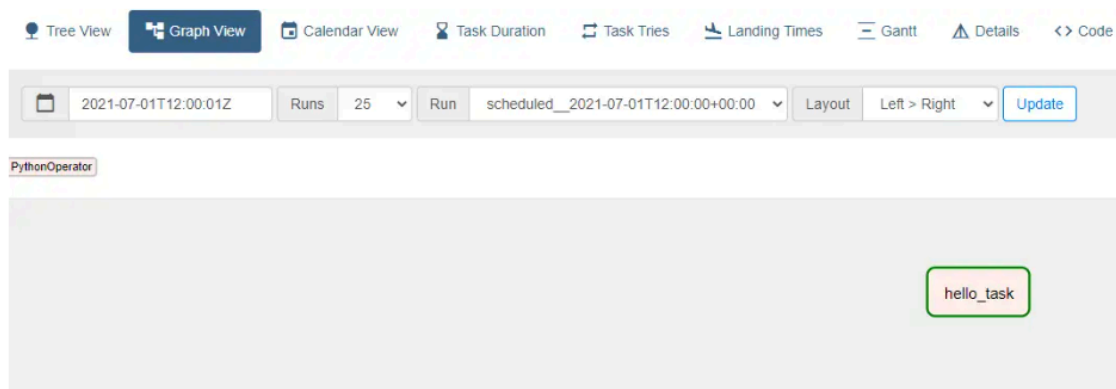
Para executar o DAG, precisamos mantê-lo ativo. No arquivo python nós já configuramos as formas de execução.

```
start_date=datetime(2023, 8, 13...
```



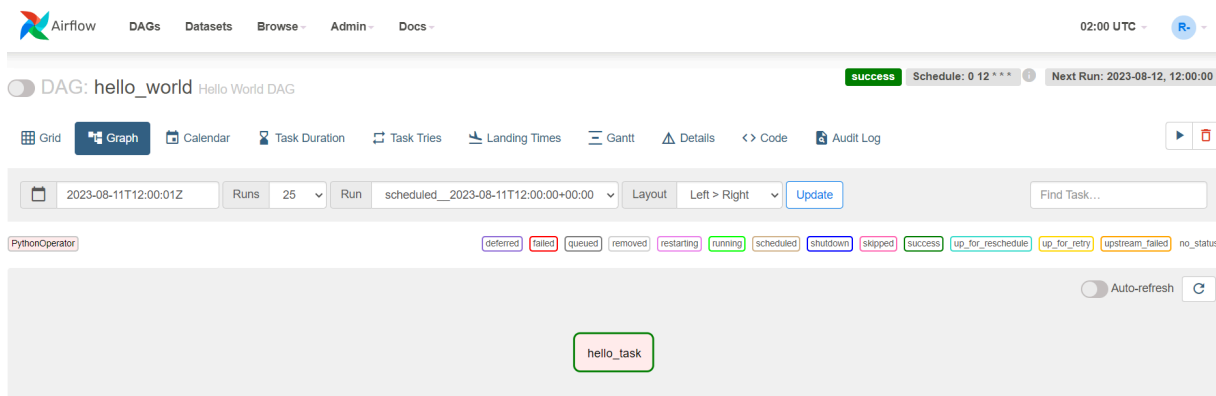


Para iniciar o DAG, podemos ativá-lo clicando no botão de alternância antes do nome do DAG. Assim que isso for feito, poderemos ver mensagens nos logs do agendador sobre a execução do DAG. Também podemos ver a visualização do gráfico DAG onde o operador `hello_world` foi executado com êxito.



Aqui, podemos ver a mensagem da DAG hello world. Em outras palavras, nosso DAG foi executado com sucesso e a tarefa foi marcada como **SUCESSO**.

Criamos com sucesso nosso primeiro DAG e o executamos usando Apache Airflow. Embora tenha sido uma mensagem de saudação simples, ela nos ajudou a entender os conceitos por trás de uma execução de DAG.



b. Subir o arquivo “dag2.py” como DAG e analisá-lo

Baixe o arquivo dag2.py de:

https://drive.google.com/drive/folders/1Efgpk4ZZNv_ZuVGam7ePavTY-PKOUe81

Avalie o que está acontecendo clicando dentro de cada **task** e observando o **Graph**.

Expandindo para Tarefas Reais

Este é apenas um primeiro passo para começar com o Apache Airflow. À medida que você se familiariza mais com a ferramenta, pode explorar recursos avançados, como variáveis, pools, sensores, operadores específicos de provedores de nuvem e muito mais. Certifique-se de consultar a documentação oficial do Apache Airflow para obter mais informações detalhadas e exemplos.

Alerta Google Cloud: você recebe 300 dólares em créditos, que podem ser facilmente consumidos ao longo do tempo.

Por isso, caso tenha usado o Google Cloud, ao terminar o trabalho, desabilite a instância que contém o Apache Airflow para evitar cobranças futuras.

2. Criando uma DAG chamando o Apache Hop

<https://hop.apache.org/manual/latest/how-to-guides/run-hop-in-apache-airflow.html>

a. Apache Airflow para executar fluxos de trabalho e pipelines do Apache Hop.

Vamos analisar mais de perto algumas coisas no DAG que usaremos. Isso parecerá muito familiar se você tiver executado fluxos de trabalho e pipelines do Apache Hop em contêineres:

Se você estiver usando o Airflow no Docker, precisará importar o DockerOperator para o DAG:

```
from airflow.operators.docker_operator import DockerOperator
```

Vamos dar uma olhada no final da tarefa Apache Hop primeiro:

```
mounts=[Mount(source='LOCAL_PATH_TO_PROJECT_FOLDER',  
target='/project', type='bind'),  
Mount(source='LOCAL_PATH_TO_ENV_FOLDER',  
target='/project-config', type='bind')]
```

A seção de montagens é onde vincularemos suas pastas de projeto e ambiente ao contêiner.

LOCAL_PATH_TO_PROJECT_FOLDER é o caminho para a pasta do projeto no sistema de arquivos local (a pasta onde você mantém o arquivo hop-config.json, a pasta de metadados e os fluxos de trabalho e pipelines). Esta pasta será montada como /project dentro do contêiner.

LOCAL_PATH_TO_ENV_FOLDER é semelhante, mas aponta para a pasta onde estão os arquivos de configuração do ambiente (JSON). Essa pasta será montada como /project-config dentro do contêiner.

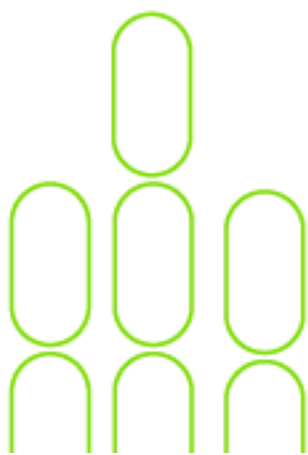


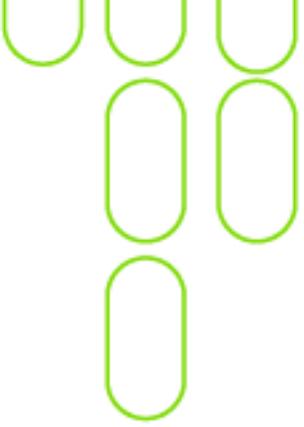
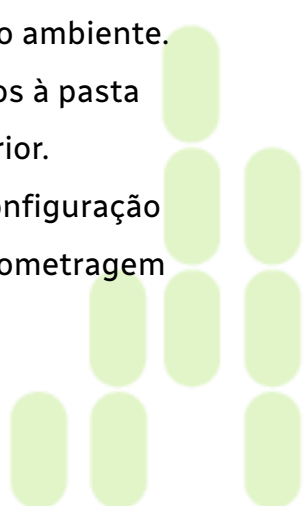
Defina e configure o pipeline em sua tarefa do DAG:

```
hop = DockerOperator(  
    task_id='sample-pipeline',  
    # use the Apache Hop Docker image. Add your tags here in the  
    default apache/hop: syntax  
    image='apache/hop',  
    api_version='auto',  
    auto_remove=True,  
    environment={  
        'HOP_RUN_PARAMETERS': 'INPUT_DIR=',  
        'HOP_LOG_LEVEL': 'Basic',  
        'HOP_FILE_PATH':  
        '${PROJECT_HOME}/transforms/null-if-basic.hpl',  
        'HOP_PROJECT_DIRECTORY': '/project',  
        'HOP_PROJECT_NAME': 'hop-airflow-sample',  
        'HOP_ENVIRONMENT_NAME': 'env-hop-airflow-sample.json',  
        'HOP_ENVIRONMENT_CONFIG_FILE_NAME_PATHS':  
        '/project-config/env-hop-airflow-sample.json',  
        'HOP_RUN_CONFIG': 'local'  
    },  
    }
```

Os parâmetros a especificar aqui são:

- **task_id:** um ID exclusivo para essa tarefa de Airflow no DAG.
- **image:** usamos "Apache/Hop" neste exemplo, que sempre pegará a versão mais recente. Adicione uma tag para usar uma versão específica do Apache Hop, por exemplo, "apache/hop:2.6.0" ou "apache/hop:Development" para a versão de desenvolvimento mais recente.



- 
- **environment** é onde informaremos ao DockerOperator qual pipeline executar e forneceremos configuração adicional. As variáveis de ambiente usadas aqui são exatamente o que você passaria para um contêiner autônomo de curta duração sem Airflow:
 - o **HOP_RUN_PARAMETERS:** parâmetros a serem passados para o fluxo de trabalho ou pipeline.
 - o **HOP_LOG_LEVEL:** nível de log a ser usado com seu fluxo de trabalho ou pipeline.
 - o **HOP_FILE_PATH:** caminho para o fluxo de trabalho ou pipeline que você deseja usar. Esse é o caminho no contêiner e é relativo à pasta do projeto.
 - o **HOP_PROJECT_DIRECTORY:** pasta onde os arquivos de projeto estão salvos. Neste exemplo, esta é a pasta /project que montamos na seção anterior.
 - o **HOP_PROJECT_NAME:** nome do projeto Apache Hop. Isso só será usado internamente (e será mostrado nos logs). O nome do seu projeto não é necessariamente o mesmo nome que você usou para desenvolver o projeto no Hop Gui, mas manter as coisas consistentes nunca é demais.
 - o **HOP_ENVIRONMENT_NAME:** semelhante ao nome do projeto, esse é o nome do ambiente que será criado por meio do hop-conf quando o contêiner for iniciado.
 - o **HOP_ENVIRONMENT_CONFIG_FILE_NAME_PATHS:** caminhos para os arquivos de configuração do ambiente. Esses caminhos de arquivo devem ser relativos à pasta /project-config que montamos na seção anterior.
 - o **HOP_RUN_CONFIG:** fluxo de trabalho ou a configuração de execução do pipeline a ser usada. Sua quilometragem
- 

pode variar, mas na grande maioria dos casos, usar uma configuração de execução local será o que você precisa.

Isso é tudo o que precisamos especificar para uma primeira execução.

Este DAG será parecido com o exemplo abaixo:

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash_operator import BashOperator
from airflow.operators.docker_operator import DockerOperator
from airflow.operators.python_operator import BranchPythonOperator
from airflow.operators.dummy_operator import DummyOperator
from docker.types import Mount

default_args = {
    'owner': 'airflow',
    'description': 'sample-pipeline',
    'depend_on_past': False,
    'start_date': datetime(2022, 1, 1),
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5)
}

with DAG('sample-pipeline', default_args=default_args,
        schedule_interval=None, catchup=False, is_paused_upon_creation=False)
    as dag:
        start_dag = DummyOperator(
            task_id='start_dag'
        )
        end_dag = DummyOperator(
            task_id='end_dag'
        )
        hop = DockerOperator(
            task_id='sample-pipeline',
            # use the Apache Hop Docker image. Add your tags here in the
            default apache/hop: syntax
            image='apache/hop',
            api_version='auto',
            auto_remove=True,
            environment= {
                'HOP_RUN_PARAMETERS': 'INPUT_DIR=',
                'HOP_LOG_LEVEL': 'Basic',
                'HOP_FILE_PATH':
                '${PROJECT_HOME}/transforms/null-if-basic.hpl',
                'HOP_PROJECT_DIRECTORY': '/project',
                'HOP_PROJECT_NAME': 'hop-airflow-sample',
                'HOP_ENVIRONMENT_NAME': 'env-hop-airflow-sample.json',
                'HOP_ENVIRONMENT_CONFIG_FILE_NAME_PATHS':
                '/project-config/env-hop-airflow-sample.json',
```

```

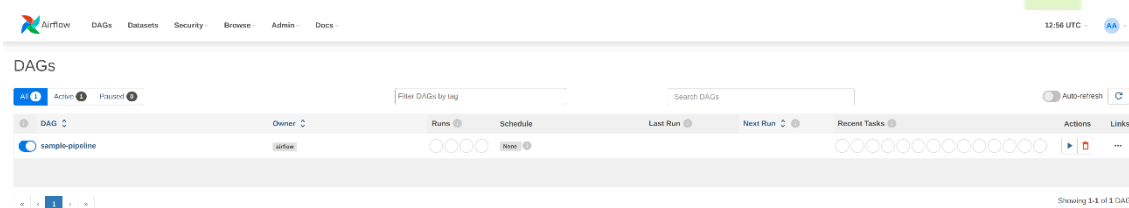
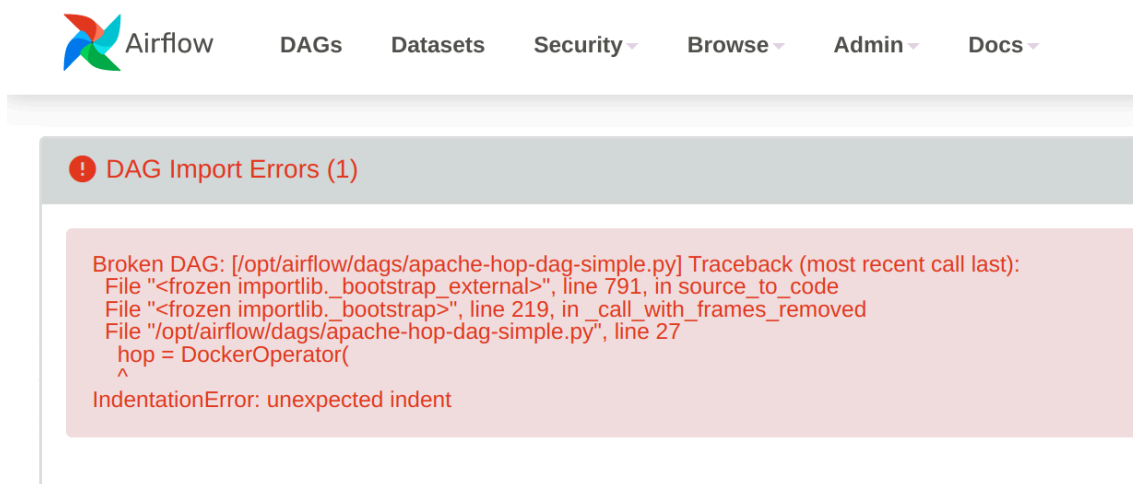
        'HOP_RUN_CONFIG': 'local'
    },
    docker_url="unix://var/run/docker.sock",
    network_mode="bridge",
    mounts=[Mount(source='LOCAL_PATH_TO_PROJECT_FOLDER',
target='/project', type='bind'),
Mount(source='LOCAL_PATH_TO_ENV_FOLDER', target='/project-config',
type='bind')],
    force_pull=False
)
start_dag >> hop >> end_dag

```

a. Publicar e executar seu DAG

Tudo o que é preciso para publicar seu dag é colocá-lo na pasta dags do Airflow. Salve o DAG que acabamos de criar em sua pasta dags como **apache-hop-dag-simple.py**. Depois de uma breve espera, seu DAG aparecerá na lista de dags.

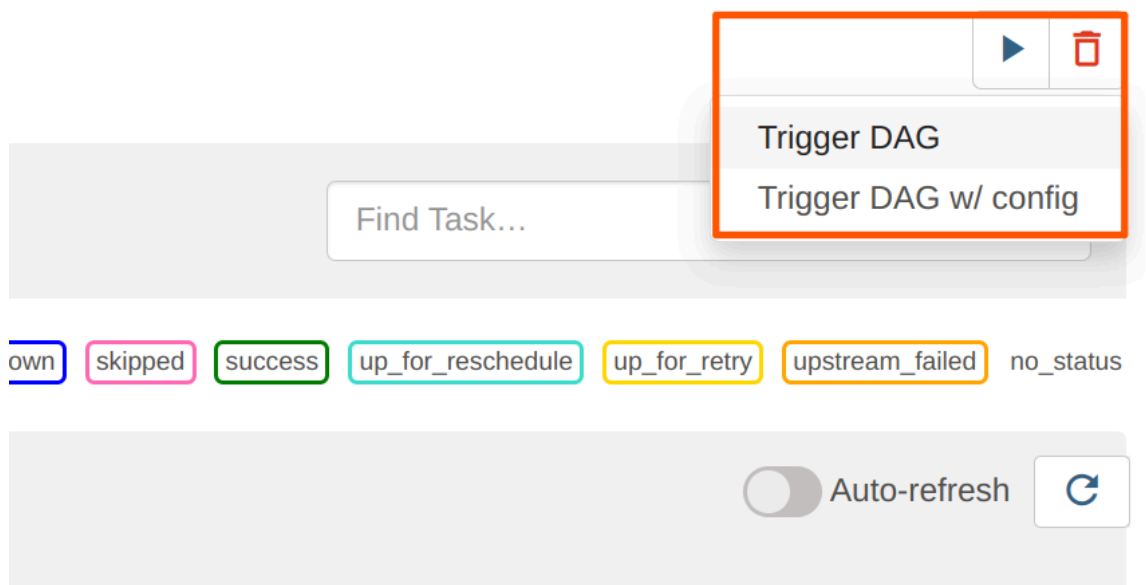
Se houver algum erro de sintaxe no seu DAG, o Airflow informará você. Expanda a caixa de diálogo de erro para obter mais detalhes sobre o erro.



No DAG, clique no botão sample-pipeline para ver mais detalhes sobre ele. Na lista de guias na parte superior da página, selecione "Código" para revisar o DAG que você acabou de implantar ou "Gráfico" para ver a representação gráfica do DAG. Este gráfico é extremamente simples, mas estamos explorando o Apache Airflow, então isso é intencional.



Para executar este DAG, clique no ícone de reprodução para a opção com o botão Trigger DAG. O ícone está disponível em vários locais na interface do usuário do Apache Airflow. Está quase sempre disponível no canto superior direito.



Seu DAG será executado em segundo plano. Para acompanhar e verificar os registros, clique no nome do seu DAG para ir para a página de detalhes.

The screenshot displays the Airflow web interface. At the top, there's a navigation bar with links for DAGs, Datasets, Security, Browse, Admin, and Docs. Below this, a filter bar shows the current time (05/08/2023, 05:48:39 AM) and a dropdown for '25' items. A 'Clear Filters' button is also present. The main content area shows the 'sample-pipeline' DAG details for the run on 2023-05-07 at 13:54:09 UTC. The 'Logs' tab is selected, showing a list of log entries. The logs indicate that the DAG was successfully executed, with tasks like 'start_dag', 'sample_pipeline', and 'end_dag' all running without errors. The logs also show the creation of a project folder and the execution of a script named 'sample.py'.

Quando você retornar à tela inicial do Airflow, seu DAG mostrará círculos verdes para execuções bem-sucedidas.

This screenshot shows the DAG 'sample-pipeline' in the Airflow web interface. The DAG is represented by a green circle, indicating a successful run. The interface includes a 'DAG' dropdown, a 'sample-pipeline' link, and a 'Runs' section showing the DAG's status as 'None'. The 'Schedule' section shows the DAG's schedule as '2023-05-07, 13:54:09'. The 'Recent Tasks' section shows a list of tasks, with the first task 'sample_pipeline' having a green circle next to it, indicating a successful run. The 'Actions' and 'Links' sections are also visible.